

第一個 WebGPU 應用程式

來源：<https://codelabs.developers.google.com/your-first-webgpu-app?hl=zh-tw>

步驟數：9

本程式碼研究室將介紹新 WebGPU API 的基礎知識。並引導您完成在 GPU 上執行的 Conway 的《Game of Life》版本。WebGPU 的轉譯功能可用來繪製主機板，WebGPU 的運算功能則可用來更新遊戲狀態。

目錄

1. [1. 簡介](#)
2. [2. 做好準備](#)
3. [3. 初始化 WebGPU](#)
4. [4. 繪製幾何圖形](#)
5. [5. 繪製格線](#)
6. [6. 加分題：讓畫面更加繽紛！](#)
7. [7. 管理儲存格狀態](#)
8. [8. 執行模擬](#)
9. [9. 恭喜！](#)

1. 簡介



什麼是 WebGPU ?

WebGPU 是新式 API，可供網頁應用程式存取 GPU 功能。

新版 API

在 WebGPU 之前，有 [WebGL](#)，但僅提供 WebGPU 的部分功能。這項技術催生了新一代的豐富網頁內容，開發人員也運用這項技術打造出許多令人驚豔的內容。不過，該 API 是以 2007 年發布的 [OpenGL ES 2.0](#) API 為基礎，而後者又是以更舊的 OpenGL API 為基礎。這段期間 GPU 的發展突飛猛進，用來與 GPU 介接的原生 API 也隨之演進，例如 [Direct3D 12](#)、[Metal](#) 和 [Vulkan](#)。

WebGPU 將這些現代 API 的進展帶入網頁平台。這項技術的重點是跨平台啟用 GPU 功能，同時提供在網路上感覺很自然的 API，而且比某些建構於其上的原生 API 更簡潔。

轉譯

GPU 通常與快速算繪詳細圖像相關聯，WebGPU 也不例外。這項 API 具備所需功能，可支援現今許多最熱門的算繪技術，適用於桌機和行動裝置 GPU，並提供路徑，讓您在硬體功能持續演進時，新增未來功能。

運算

除了算繪之外，WebGPU 還能發揮 GPU 的潛力，執行一般用途的高度平行工作負載。這些 [運算著色器](#) 可單獨使用，不需任何算繪元件，也可以緊密整合至算繪管線。

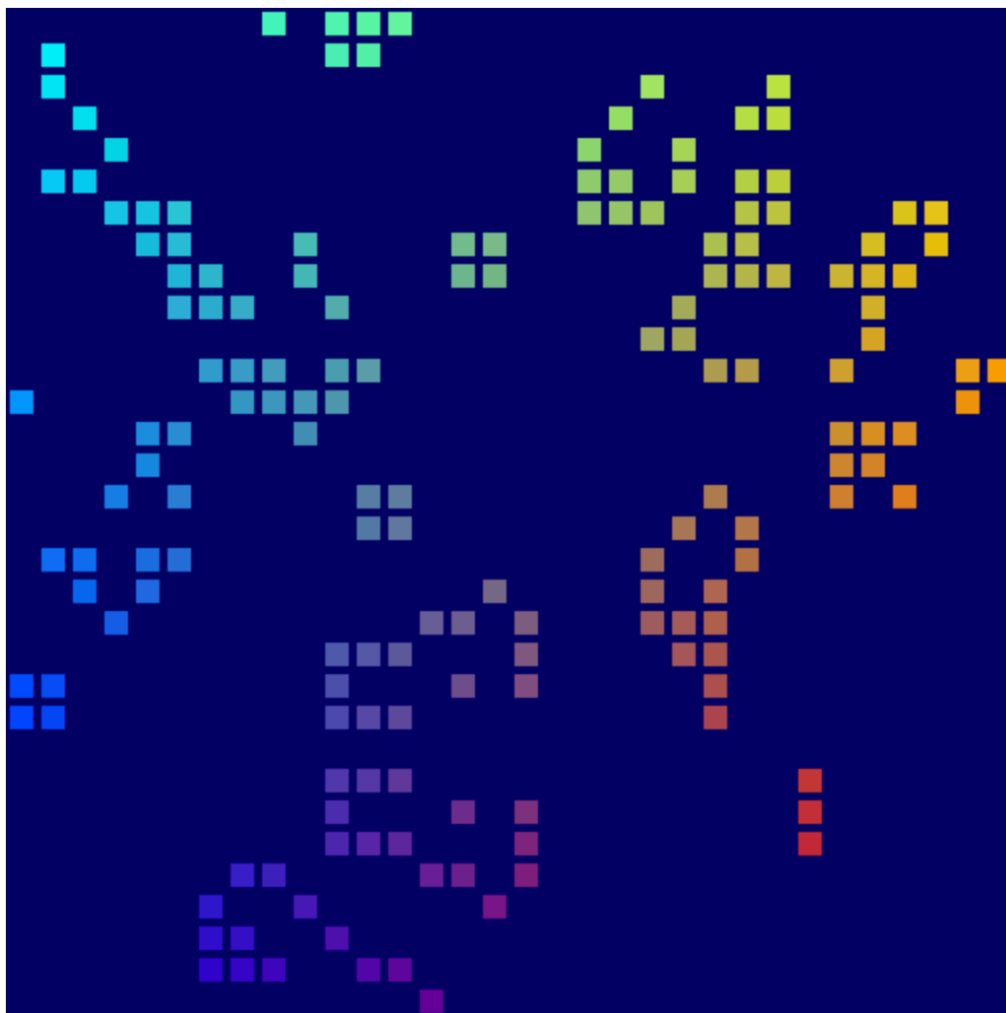
在本程式碼研究室中，您將瞭解如何運用 WebGPU 的算繪和運算功能，建立簡單的入門專案！

建構項目

在本程式碼研究室中，您將使用 WebGPU 建構 [康威生命遊戲](#)。您的應用程式將會：

- 使用 WebGPU 的算繪功能繪製簡單的 2D 圖像。

- 使用 WebGPU 的運算功能執行模擬作業。



注意：為簡化本程式碼研究室，所有程式碼片段都是純 JavaScript。WebGPU 可與 [TypeScript](#) 完美搭配，但您需要加入 [@webgpu/types](#) 定義。API 獲得更廣泛的實作後，這些型別應會成為標準 TypeScript DOM 和工作站程式庫的一部分。

生命遊戲是一種細胞自動機，其中的細胞格會根據一組規則隨時間改變狀態。在生命遊戲中，儲存格會根據相鄰儲存格的啟用狀態而啟用或停用，因此會產生有趣的模式，並在您觀看時不斷變化。

課程內容

- 如何設定 WebGPU 及設定畫布。
- 如何繪製簡單的 2D 幾何圖形。
- 如何使用頂點和片段著色器，修改繪製內容。
- 如何使用運算著色器執行簡單的模擬作業。

本程式碼研究室著重於介紹 WebGPU 的基本概念。本文並非要全面檢視 API，也不會涵蓋 (或要求) 3D 矩陣數學等常見相關主題。

軟硬體需求

- 在 ChromeOS、macOS 或 Windows 上使用新版 Chrome (113 以上版本)。WebGPU 是跨瀏覽器和跨平台的 API，但尚未在所有地方推出。
- 熟悉 HTML、JavaScript 和 [Chrome 開發人員工具](#)。

您不需要熟悉其他 Graphics API，例如 WebGL、Metal、Vulkan 或 Direct3D，但如果您有相關經驗，可能會發現 WebGPU 與這些 API 有許多相似之處，有助於快速入門！

步驟 2 · 約 2 分鐘

2. 做好準備

取得程式碼

本程式碼研究室沒有任何依附元件，而且會逐步引導您建立 WebGPU 應用程式，因此您不需要任何程式碼即可開始。不過，您可以在 <https://github.com/GoogleChromeLabs/your-first-webgpu-app-codelab> 找到一些可做為查核點的有效範例。如果遇到困難，可以查看這些範例並做為參考。

使用開發人員控制台！

WebGPU 是相當複雜的 API，有許多規則會強制執行正確用法。更糟的是，由於 API 的運作方式，許多錯誤無法引發典型的 JavaScript 例外狀況，因此更難準確找出問題來源。

使用 WebGPU 開發時，您會遇到問題，尤其是初學者，這很正常！API 背後的開發人員瞭解 GPU 開發的挑戰，因此努力確保每當 WebGPU 程式碼導致錯誤時，您都會在開發人員控制台中收到非常詳細且實用的訊息，協助您找出並修正問題。

在處理任何網頁應用程式時，開啟控制台一律很有用，但這裡尤其適用！

3. 初始化 WebGPU

首先請參加 `<canvas>`

如果只想使用 WebGPU 進行運算，不必在畫面上顯示任何內容。但如果您想算繪任何內容 (例如本程式碼研究室要執行的操作)，就需要畫布。因此這是個不錯的起點！

建立新的 HTML 文件，其中包含單一 `<canvas>` 元素，以及我們查詢畫布元素的 `<script>` 標記。(或使用 [00-starter-page.html](#))。

- 建立 `index.html` 檔案並加入以下程式碼：

index.html

```
<!doctype html>

<html>
  <head>
    <meta charset="utf-8">
    <title>WebGPU Life</title>
  </head>
  <body>
    <canvas width="512" height="512"></canvas>
    <script type="module">
      const canvas = document.querySelector("canvas");

      // Your WebGPU code will begin here!
    </script>
  </body>
</html>
```

注意：畫布元素的預設大小為 300x150 像素，對這個應用程式來說太小了。雖然不一定要使用 WebGPU，但您可能想為畫布設定較大的寬度和高度。上述範例程式碼會將畫布設為 512x512 CSS 像素，但您可以自由選擇所需大小。

注意：將指令碼標記的類型設為 `"module"`，即可使用[頂層 await](#)，這對 WebGPU 初始化作業很有幫助！

要求轉接頭和裝置

現在可以開始瞭解 WebGPU 位元了！首先，您應該考慮到 WebGPU 等 API 可能需要一段時間，才能在整個網路生態系統中傳播。因此，建議您先檢查使用者的瀏覽器是否支援 WebGPU。

1. 如要檢查做為 WebGPU 進入點的 `navigator.gpu` 物件是否存在，請新增下列程式碼：

index.html

```
if (!navigator.gpu) {
  throw new Error("WebGPU not supported on this browser.");
}
```

理想情況下，如果 WebGPU 無法使用，您會希望網頁改用不使用 WebGPU 的模式，藉此通知使用者。(或許可以改用 WebGL?) 不過，在本程式碼研究室中，您只要擲回錯誤，即可停止進一步執行程式碼。

確認瀏覽器支援 WebGPU 後，初始化應用程式 WebGPU 的第一步是要求 `GPUAdapter`。您可以將介面卡視為 WebGPU 在裝置中特定 GPU 硬體的代表。

2. 如要取得轉接程式，請使用 `navigator.gpu.requestAdapter()` 方法。這個函式會傳回 Promise，因此最方便的做法是使用 `await` 呼叫函式。

index.html

```
const adapter = await navigator.gpu.requestAdapter();
if (!adapter) {
  throw new Error("No appropriate GPUAdapter found.");
}
```

如果找不到適當的轉接器，傳回的 `adapter` 值可能是 `null`，因此您需要處理這種情況。如果使用者的瀏覽器支援 WebGPU，但 GPU 硬體不具備使用 WebGPU 的所有必要功能，就可能會發生這種情況。

在大多數情況下，只要讓瀏覽器挑選預設的轉接器即可，就像您在這裡所做的一樣，但如有更進階的需求，可以將引數傳遞至 `requestAdapter()`，指定要在有多個 GPU 的裝置 (例如某些筆電) 上使用低功耗或高效能的硬體。

取得介面卡後，您需要先要求 `GPUDevice`，才能開始使用 GPU。裝置是與 GPU 互動的主要介面。

3. 呼叫 `adapter.requestDevice()` 即可取得裝置，這個方法也會傳回 Promise。

index.html

```
const device = await adapter.requestDevice();
```

與 `requestAdapter()` 相同，這裡也有可傳遞的選項，可用於啟用特定硬體功能或要求提高限制等進階用途，但就您的目的而言，預設值就已足夠。

設定 Canvas

現在您已擁有裝置，如果想在網頁上顯示任何內容，還有一件事要做：設定要與您剛建立的裝置搭配使用的畫布。

- 如要執行這項操作，請先呼叫 `canvas.getContext("webgpu")`，從畫布要求 `GPUCanvasContext`。(這與您用來初始化 Canvas 2D 或 WebGL 環境的呼叫相同，分別使用 `2d` 和 `webgl` 環境類型)。然後，傳回的 `context` 必須使用 `configure()` 方法與裝置建立關聯，如下所示：

index.html


```
const context = canvas.getContext("webgpu");
const canvasFormat = navigator.gpu.getPreferredCanvasFormat();
context.configure({
  device: device,
  format: canvasFormat,
});
```

這裡可以傳遞幾個選項，但最重要的選項是您要搭配使用的內容 `device`，以及內容應使用的紋理格式 `format`。

紋理是 WebGPU 用來儲存圖片資料的物件，每個紋理都有格式，可讓 GPU 瞭解資料在記憶體中的配置方式。紋理記憶體的運作方式不在本程式碼研究室的範圍內。請務必瞭解畫布內容會提供紋理，供程式碼繪製內容，而您使用的格式可能會影響畫布顯示這些圖像的效率。不同類型的裝置最適合使用不同的紋理格式，如果沒有使用裝置偏好的格式，可能會導致圖片在顯示為網頁的一部分之前，在背景發生額外的記憶體副本。

幸好，您不必擔心這些問題，因為 WebGPU 會告訴您要為畫布使用哪種格式！在幾乎所有情況下，您都會想傳遞呼叫 `navigator.gpu.getPreferredCanvasFormat()` 傳回的值，如上所示。

注意：WebGPU 與 WebGL 的一大差異在於，畫布設定與裝置建立作業是分開的，因此您可以擁有任意數量的畫布，全部由單一裝置算繪！這樣一來，開發多窗格 3D 編輯器等特定用途的應用程式就會容易許多。

清除畫布

現在您已擁有裝置，且畫布已設定完成，可以開始使用裝置變更畫布內容。首先，請使用單一顏色清除畫面。

如要執行這項操作 (或 WebGPU 中的任何其他操作)，您必須向 GPU 提供一些指令，指示要執行的動作。

1. 如要執行這項操作，請讓裝置建立 `GPUCommandEncoder`，這個物件會提供記錄 GPU 指令的介面。

index.html

```
const encoder = device.createCommandEncoder();
```

您要傳送至 GPU 的指令與算繪相關 (在本例中為清除畫布)，因此下一步是使用 `encoder` 開始算繪通道。

Render Pass 是指 WebGPU 中所有繪圖作業發生的時間。每個呼叫都會以 `beginRenderPass()` 開頭，定義接收任何繪圖指令輸出的紋理。進階用途可提供多種紋理 (稱為「附件」)，用途包括儲存算繪幾何的深度或提供反鋸齒。不過，這個應用程式只需要一個。

2. 呼叫 `context.getCurrentTexture()`，從先前建立的畫布內容取得材質，這會傳回材質，其像素寬度和高度與畫布的 `width` 和 `height` 屬性，以及您呼叫 `context.configure()` 時指定的 `format` 相符。

index.html

```
const pass = encoder.beginRenderPass({
  colorAttachments: [{
    view: context.getCurrentTexture().createView(),
    loadOp: "clear",
    storeOp: "store",
  }]
});
```

紋理會以 `colorAttachment` 的 `view` 屬性形式提供。算繪通道需要您提供 `GPUTextureView`，而非 `GPUTexture`，以便瞭解要算繪紋理的哪些部分。這只對進階用途有實際意義，因此您可以在紋理上呼叫 `createView()`，且不帶任何引數，表示您希望算繪通道使用整個紋理。

您也必須指定要在算繪通道開始和結束時，對紋理執行的動作：

- `loadOp` 值為 `"clear"` 表示您希望在算繪通道啟動時清除紋理。
- `storeOp` 值為 `"store"` 表示您希望在算繪通道完成後，將算繪通道期間完成的任何繪圖結果儲存到紋理中。

算繪通道開始後，您就... 什麼都不用做！至少目前是這樣。只要使用 `loadOp: "clear"` 啟動算繪通道，即可清除紋理檢視區塊和畫布。

3. 在 `beginRenderPass()` 後方立即新增下列呼叫，結束算繪通道：

index.html

```
pass.end();
```

請務必瞭解，單純進行這些呼叫不會導致 GPU 實際執行任何動作。這些指令只是記錄下來，供 GPU 稍後執行。

4. 如要建立 `GPUCommandBuffer`，請在指令編碼器上呼叫 `finish()`。指令緩衝區是不透明的控制代碼，用於記錄指令。

index.html

```
const commandBuffer = encoder.finish();
```

5. 使用 `GPUDevice` 的 `queue` 將指令緩衝區提交至 GPU。佇列會執行所有 GPU 指令，確保指令執行順序正確，並妥善同步處理。佇列的 `submit()` 方法會接收指令緩衝區陣列，但本例中只有一個。

index.html

```
device.queue.submit([commandBuffer]);
```

提交指令緩衝區後就無法再次使用，因此不需要保留。如要提交更多指令，請建構另一個指令緩衝區。因此，這兩個步驟通常會合併為一個，就像本程式碼研究室的範例網頁一樣：

index.html

```
// Finish the command buffer and immediately submit it.
device.queue.submit([encoder.finish()]);
```

將指令提交至 GPU 後，請讓 JavaScript 將控制權交還給瀏覽器。此時，瀏覽器會發現您已變更內容的目前紋理，並更新畫布，將該紋理顯示為圖片。如要在之後再次更新畫布內容，您必須錄製並提交新的指令緩衝區，再次呼叫 `context.getCurrentTexture()`，取得算繪通道的新紋理。

6. 重新載入頁面。請注意，畫布會填滿黑色。恭喜！這表示您已成功建立第一個 WebGPU 應用程式。



挑選顏色！

不過說實話，黑色方塊相當無趣。因此，請先花點時間稍微調整一下，再繼續閱讀下一節。

1. 在 `encoder.beginRenderPass()` 呼叫中，將含有 `clearValue` 的新行加到 `colorAttachment`，如下所示：

index.html

```
const pass = encoder.beginRenderPass({
  colorAttachments: [{
    view: context.getCurrentTexture().createView(),
    loadOp: "clear",
    clearValue: { r: 0, g: 0, b: 0.4, a: 1 }, // New line
    storeOp: "store",
  }],
});
```

`clearValue` 會指示算繪通道在通道開始時執行 `clear` 作業時應使用的顏色。傳遞至其中的字典包含四個值：`r` 代表紅色、`g` 代表綠色、`b` 代表藍色，以及 `a` 代表 *alpha* (透明度)。每個值介於 0 到 1 之間，共同描述該顏色通道的值。例如：

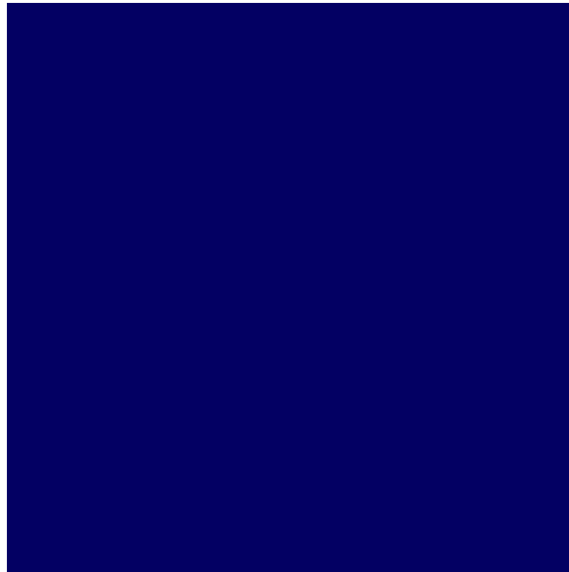
- `{ r: 1, g: 0, b: 0, a: 1 }` 為亮紅色。
- `{ r: 1, g: 0, b: 1, a: 1 }` 為亮紫色。
- `{ r: 0, g: 0.3, b: 0, a: 1 }` 為深綠色。
- `{ r: 0.5, g: 0.5, b: 0.5, a: 1 }` 為中灰色。
- 預設值為透明黑色 `{ r: 0, g: 0, b: 0, a: 0 }`。

注意：在 WebGPU 中定義顏色與使用 `rgb()` 函式標記設定 CSS 顏色非常相似。

專業提示：如要減少輸入內容，也可以使用陣列簡寫，依 RGBA 順序提供值。傳遞 `[0, 0.5, 0.7, 1]` 與傳遞 `{ r: 0, g: 0.5, b: 0.7, a: 1 }` 相同。

本程式碼實驗室的範例程式碼和螢幕截圖使用深藍色，但您可以隨意選擇喜歡的顏色！

2. 選取顏色後，請重新載入頁面。畫布中應該會顯示你選擇的顏色。



4. 繪製幾何圖形

在本節結束時，您的應用程式會在畫布上繪製一些簡單的幾何圖形：彩色正方形。請注意，雖然輸出內容很簡單，但您可能會覺得工作量很大，這是因為 WebGPU 的設計宗旨是大量幾何圖形的高效率算繪。這種效率的副作用是，相對簡單的事情可能會感覺異常困難，但如果您使用 WebGPU 這類 API，就應該會預期到這種情況，因為您想做的事情稍微複雜一些。

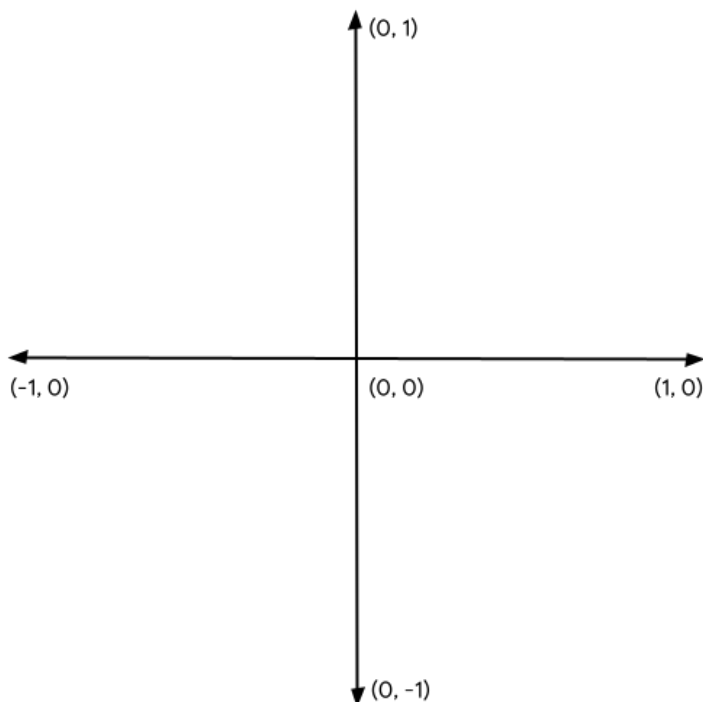
瞭解 GPU 的繪圖方式

在進行任何程式碼變更之前，建議先快速簡化地概略瞭解 GPU 如何在螢幕上建立形狀。(如果您已熟悉 GPU 算繪的基本運作方式，可以略過本節，直接前往「定義頂點」一節)。

與 Canvas 2D 等 API 不同，GPU 實際上只處理幾種不同類型的形狀 (或 WebGPU 所稱的「基本體」*primitives*)：點、線和三角形。在本程式碼研究室中，您只會使用三角形。

GPU 幾乎只處理三角形，因為三角形有許多良好的數學特性，可讓 GPU 以可預測且有效率的方式處理。GPU 繪製的幾乎所有內容都必須先分割成三角形，且這些三角形必須由角點定義。

這些點或頂點會以 X、Y 和 (3D 內容的) Z 值表示，定義 WebGPU 或類似 API 所定義的[笛卡兒座標系統](#)上的點。座標系統的結構最容易從與網頁畫布的關係來思考。無論畫布有多寬或多高，左側邊緣的 X 軸一律為 -1，右側邊緣的 X 軸一律為 +1。同樣地，Y 軸上的底邊一律為 -1，頂邊則為 +1。也就是說，(0, 0) 一律是畫布中心，(-1, -1) 一律是左下角，(1, 1) 一律是右上角。這就是所謂的「剪輯空間」。



頂點很少會在一開始就定義在這個座標系統中，因此 GPU 會依賴稱為「頂點著色器」的小型程式，執行將頂點轉換為裁剪空間所需的任何數學運算，以及繪製頂點所需的任何其他計算。舉例來說，著色器可能會套用某些動畫，或計算從頂點到光源的方向。這些著色器是由 WebGPU 開發人員編寫，可大幅控管 GPU 的運作方式。

接著，GPU 會取得這些轉換後的頂點所組成的所有三角形，並判斷需要繪製哪些螢幕上的像素。接著，系統會執行您撰寫的另一個小型程式 (稱為「片段著色器」)，計算每個像素應有的顏色。計算結果可以簡單到 *return green*，也可以複雜到計算表面相對於其他附近表面反射的陽光的角度，並透過霧氣過濾，以及根據表面的金屬程度進行修改。這完全由您掌控，既能賦予力量，也可能令人不知所措。

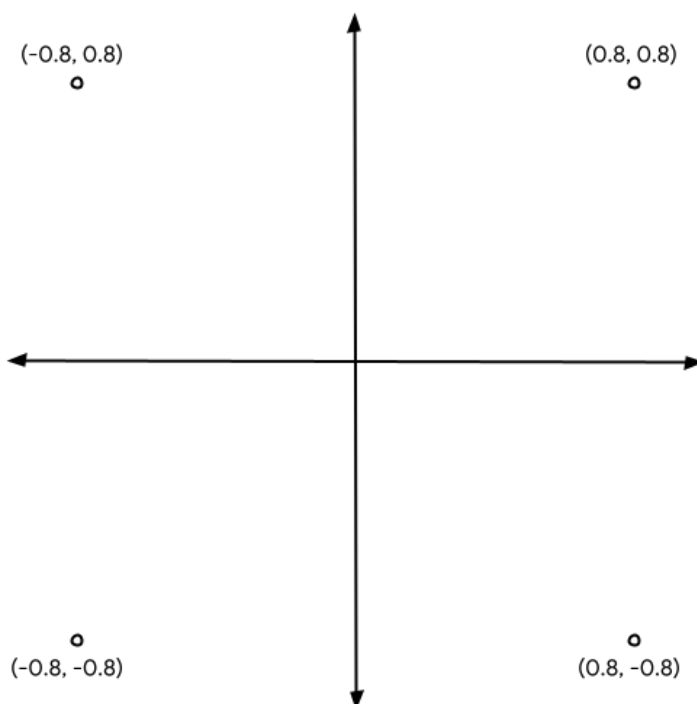
接著，系統會將這些像素顏色的結果累積到紋理中，然後顯示在畫面上。

注意：本程式碼研究室只會處理 2D 形狀，但這個程序也適用於 3D 內容！GPU 的 2D 和 3D 有何不同？只要多做一點數學運算即可！3D 內容通常會使用一系列矩陣，在繪製三角形之前轉換頂點著色器中的位置 (稍後會介紹)，藉此呈現深度和體積的感覺，但最終 GPU 繪製的幾乎所有內容都只是剪輯空間中的三角形。

定義頂點

如先前所述，生命遊戲模擬會以儲存格格線顯示。您的應用程式需要以某種方式顯示格線，區分有效儲存格和無效儲存格。本程式碼研究室採用的方法是在有效儲存格中繪製彩色方塊，並將無效儲存格留空。

也就是說，您必須為 GPU 提供四個不同的點，分別對應正方形的四個角。舉例來說，在畫布中央繪製的正方形，從邊緣拉入一段距離後，會有如下的角座標：



如要將這些座標提供給 GPU，您必須將值放在 `TypedArray` 中。如果您還不熟悉 `TypedArrays`，這是一組 JavaScript 物件，可讓您分配連續的記憶體區塊，並將序列中的每個元素解讀為特定資料型別。舉例來說，在 `Uint8Array` 中，陣列中的每個元素都是單一的無號位元組。`TypedArray` 非常適合與對記憶體配置很敏感的 API 來回傳送資料，例如 `WebAssembly`、`WebAudio` 和 (當然) `WebGPU`。

以正方形為例，由於值是分數，因此適合使用 `Float32Array`。

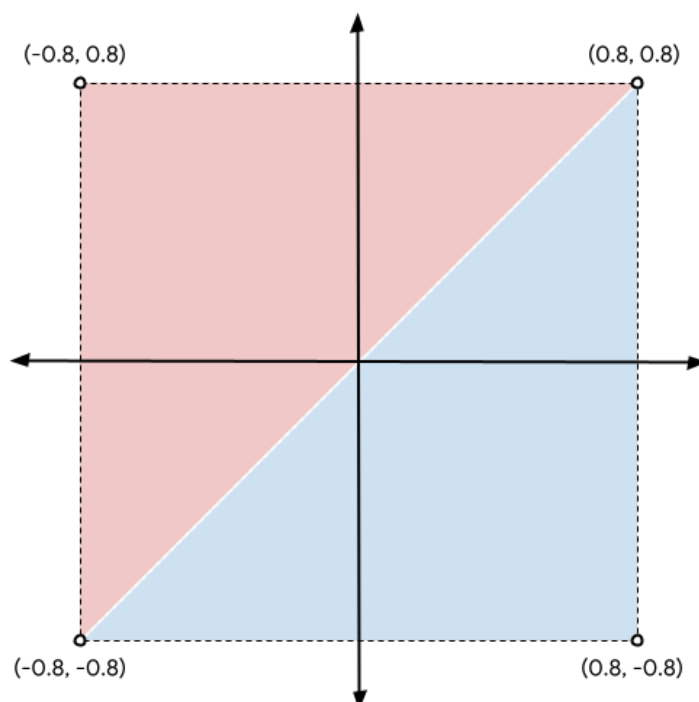
1. 在程式碼中加入下列陣列宣告，建立陣列來保存圖表中的所有頂點位置。建議將其放在頂端附近，也就是 `context.configure()` 呼叫下方。

index.html

```
const vertices = new Float32Array([
  //   X,    Y,
  -0.8, -0.8,
   0.8, -0.8,
   0.8,  0.8,
  -0.8,  0.8,
]);
```

請注意，間距和註解不會影響值，只是為了方便您閱讀。這有助於您瞭解每對值如何構成一個頂點的 X 和 Y 座標。

但有個問題！還記得嗎？GPU 是以三角形為單位運作，因此，您必須以三組為單位提供頂點。你有一組四人。解決方法是重複兩個頂點，在正方形中間建立共用邊的兩個三角形。



如要從圖表形成正方形，您必須列出 (-0.8, -0.8) 和 (0.8, 0.8) 頂點兩次，藍色三角形和紅色三角形各一次。(您也可以選擇以其他兩個角分割正方形，結果並無差異)。

2. 更新先前的 `vertices` 陣列，內容大致如下：

index.html

```
const vertices = new Float32Array([
  //   X,     Y,
  -0.8, -0.8, // Triangle 1 (Blue)
    0.8, -0.8,
    0.8,  0.8,

  -0.8, -0.8, // Triangle 2 (Red)
    0.8,  0.8,
   -0.8,  0.8,
]);
```

雖然圖表為求明確而將兩個三角形分開，但頂點位置完全相同，GPU 會算繪這些三角形，不會有間隙。系統會將其算繪為單一實心方塊。

注意：您不必重複頂點資料，即可建立三角形。您可以使用稱為「索引緩衝區」的項目，將獨立的值清單饋送至 GPU，告知要將哪些頂點連在一起形成三角形，這樣就不必重複這些頂點。就像是連點成線！由於頂點資料非常簡單，本程式碼研究室不會使用索引緩衝區。但對於更複雜的幾何圖形，這絕對是值得使用的功能。

建立頂點緩衝區

GPU 無法使用 JavaScript 陣列中的資料繪製頂點。GPU 通常有自己的記憶體，且經過高度最佳化，適合用於算繪，因此您想讓 GPU 在繪製時使用的任何資料，都必須放在該記憶體中。

對於許多值 (包括頂點資料)，GPU 端記憶體是透過 `GPUBuffer` 物件管理。緩衝區是 GPU 容易存取的記憶體區塊，並標示特定用途。您可以將其視為 GPU 可見的 `TypedArray`。

1. 如要建立緩衝區來保存頂點，請在 `vertices` 陣列定義後，將下列呼叫新增至 `device.createBuffer()`。

index.html

```
const vertexBuffer = device.createBuffer({
  label: "Cell vertices",
  size: vertices.byteLength,
  usage: GPUBufferUsage.VERTEX | GPUBufferUsage.COPY_DST,
});
```

首先，請注意您為緩衝區提供標籤。您可以為建立的每個 WebGPU 物件加上選用標籤，而且強烈建議您這麼做！標籤可以是任何字串，只要能協助您識別物件即可。如果發生任何問題，WebGPU 產生的錯誤訊息會使用這些標籤，協助您瞭解錯誤原因。

接著，以位元組為單位指定緩衝區的 `size`。您需要 48 位元組的緩衝區，這是將 32 位元浮點數的大小 (4 個位元組) 乘以 `vertices` 陣列中的浮點數 (12) 所得的結果。幸好，`TypedArrays` 會為您計算 `byteLength`，因此您可以在建立緩衝區時使用該值。

注意：請熟悉這類位元組大小的計算。使用 GPU 需要相當多的配額！

最後，您需要指定緩衝區的用途。這是一或多個 `GPUBufferUsage` 旗標，多個旗標會與 `|` (按位元 OR) 運算子合併。在本例中，您指定要將緩衝區用於頂點資料 (`GPUBufferUsage.VERTEX`)，並希望能夠將資料複製到緩衝區 (`GPUBufferUsage.COPY_DST`)。

傳回給您的緩衝區物件是不透明的，您無法 (輕鬆) 檢查其中保存的資料。此外，大多數屬性都不可變動，也就是說，`GPUBuffer` 建立後就無法調整大小，也無法變更使用情形標記。但可以變更記憶體內容。

緩衝區最初建立時，所含記憶體會初始化為零。變更內容的方法有很多種，但最簡單的方式是使用要複製的 `TypedArray` 呼叫 `device.queue.writeBuffer()`。

2. 如要將頂點資料複製到緩衝區的記憶體，請新增下列程式碼：

index.html

```
device.queue.writeBuffer(vertexBuffer, /*bufferOffset=*/0, vertices);
```

定義頂點版面配置

現在您已擁有包含頂點資料的緩衝區，但就 GPU 而言，這只是一連串的位元組。如要使用這項工具繪製任何內容，請提供更多資訊。您需要向 WebGPU 提供更多有關頂點資料結構的資訊。

- 使用 `GPUVertexBufferLayout` 字典定義頂點資料結構：

index.html

```
const vertexBufferLayout = {
  arrayStride: 8,
  attributes: [{
    format: "float32x2",
    offset: 0,
    shaderLocation: 0, // Position, see vertex shader
  }],
};
```

乍看之下可能有點複雜，但其實很容易理解。

首先要提供的是 `arrayStride`。這是 GPU 在緩衝區中尋找下一個頂點時，需要向前跳過的位元組數。正方形的每個頂點都由兩個 32 位元浮點數組成。如前所述，32 位元浮點數為 4 個位元組，因此兩個浮點數為 8 個位元組。

接著是 `attributes` 屬性，這是一個陣列。屬性是編碼到每個頂點的個別資訊。您的頂點只包含一個屬性 (頂點位置)，但更進階的用途通常會有多個屬性的頂點，例如頂點的顏色或幾何表面指向的方向。不過，這不在本程式碼研究室的範圍內。

在單一屬性中，您首先要定義資料的 `format`。這份清單列出 `GPUVertexFormat` 類型，說明 GPU 可瞭解的各類頂點資料。您的每個頂點都有兩個 32 位元浮點數，因此您會使用 `float32x2` 格式。舉例來說，如果頂點資料是由四個 16 位元不帶正負號的整數組成，您就會改用 `uint16x4`。看出規律了嗎？

接著，`offset` 說明這個特定屬性在頂點中開始的位元組數。只有在緩衝區有多個屬性時，才需要擔心這個問題，而這在本程式碼研究室中不會發生。

最後是 `shaderLocation`。這是介於 0 到 15 之間的任意數字，且您定義的每個屬性都必須有專屬的數字。這個屬性會連結至頂點著色器中的特定輸入內容，您將在下一節中瞭解這項屬性。

請注意，雖然您現在定義這些值，但目前還不會將這些值傳遞至任何 WebGPU API。我們稍後會介紹這些值，但最簡單的做法是在定義頂點時一併考量這些值，因此現在先設定這些值，以供日後使用。

從著色器開始

現在您已擁有要算繪的資料，但仍需明確告知 GPU 如何處理這些資料。這大部分是透過著色器完成。

著色器是您編寫的小程式，會在 GPU 上執行。每個著色器都會在資料的不同階段運作：頂點處理、片段處理或一般運算。由於這些著色器位於 GPU 上，因此結構比一般 JavaScript 更嚴格。但這種結構可讓他們快速執行作業，而且最重要的是，還能平行作業！

WebGPU 中的著色器是以稱為 **WGSL** (WebGPU 著色語言) 的著色語言編寫。在語法方面，WGSL 有點類似 Rust，並提供多項功能，可讓常見的 GPU 工作 (例如向量和矩陣數學) 更輕鬆快速地完成。本程式碼研究室無法涵蓋整個著色器語言，但希望您能透過一些簡單的範例，瞭解部分基礎知識。

著色器本身會以字串形式傳遞至 WebGPU。

- 複製下列程式碼並貼到 `vertexBufferLayout` 下方的程式碼中，建立輸入著色器程式碼的位置：

index.html

```
const cellShaderModule = device.createShaderModule({
  label: "Cell shader",
  code: `
    // Your shader code will go here
  `;
});
```

如要建立您呼叫的著色器，請提供選用的 `label` 和 WGSL `code` 做為字串。`device.createShaderModule()` (請注意，您在此使用反引號，允許多行字串！) 新增一些有效的 WGSL 程式碼後，函式會傳回包含已編譯結果的 `GPUShaderModule` 物件。

注意：WGSL 著色器具有相當嚴格的編譯器，可強制執行強型別值等項目，並消除未定義的行為，這對跨平台相容性而言非常重要。因此，在建構自己的 WebGPU 應用程式時，您一定會遇到著色器編譯器錯誤！不過，請不要太過擔心。與 WebGPU 的其他部分一樣，如果發生錯誤，WGSL 編譯器會在開發人員控制台中列印非常詳細的訊息。

定義頂點著色器

從頂點著色器開始，因為 GPU 也是從這裡開始！

頂點著色器定義為函式，GPU 會針對 `vertexBuffer` 中的每個頂點呼叫該函式一次。由於 `vertexBuffer` 有六個位置 (頂點)，因此您定義的函式會呼叫六次。每次呼叫時，系統都會將 `vertexBuffer` 中的不同位置做為引數傳遞至函式，而頂點著色器函式的工作，就是傳回剪輯空間中的對應位置。

請務必瞭解，這些回呼不一定會依序呼叫。GPU 擅長平行執行這類著色器，可同時處理數百 (甚至數千！) 個頂點。這也是 GPU 速度驚人的主要原因，但同時也有限制。為確保極度平行處理，頂點著色器無法彼此通訊。每次調用著色器時，只能查看單一頂點的資料，且只能輸出單一頂點的值。

在 WGSL 中，您可以為頂點著色器函式指定任何名稱，但必須在函式前方加上 `@vertex` 屬性，指出函式代表的著色器階段。WGSL 會使用 `fn` 關鍵字表示函式，並使用括號宣告任何引數，以及使用大括號定義範圍。

1. 建立空白的 `@vertex` 函式，如下所示：

index.html (createShaderModule 程式碼)

```
@vertex
fn vertexMain() {

}
```

不過，這並無效，因為頂點著色器至少必須傳回所處理頂點在裁剪空間中的最終位置。這項資訊一律以 4 維向量的形式提供。向量在著色器中非常常見，因此在語言中會視為第一級原始型別，並有自己的型別，例如 4 維向量的 `vec4f`。2D 向量 (`vec2f`) 和 3D 向量 (`vec3f`) 也有類似型別！

2. 如要指出傳回的值是必要位置，請使用 `@builtin(position)` 屬性標記該值。`->` 符號用於表示函式傳回的內容。

index.html (createShaderModule 程式碼)

```
@vertex
fn vertexMain() -> @builtin(position) vec4f {

}
```

當然，如果函式有傳回類型，您就需要在函式主體中實際傳回值。您可以使用 `vec4f(x, y, z, w)` 語法建構要傳回的新 `vec4f`。x、y 和 z 值都是浮點數，在傳回值中表示頂點在裁剪空間中的位置。

注意：那麼 w 是什麼？w 值是三維同質頂點的第四個元件。這個說法讓您感到困惑嗎？但沒關係！您不必太過擔心。

就實務而言，有了 w 值，就能使用 4x4 矩陣進行數學運算，這在算繪 3D 圖像時很常見，但您很少需要直接操控這個值。在本程式碼研究室中，您只要將其保留為 1 即可。

3. 傳回 (0, 0, 0, 1) 的靜態值，您在技術上就擁有有效的頂點著色器，但由於 GPU 會辨識出產生的三角形只是一個點，然後捨棄該點，因此這個著色器永遠不會顯示任何內容。

index.html (createShaderModule 程式碼)

```
@vertex
fn vertexMain() -> @builtin(position) vec4f {
    return vec4f(0, 0, 0, 1); // (X, Y, Z, W)
}
```

您想改為使用所建立緩衝區的資料，因此請為函式宣告引數，並使用與 `vertexBufferLayout` 中描述內容相符的 `@location()` 屬性和型別。您指定了 `shaderLocation` 的 0，因此在 WGSL 程式碼中，請使用 `@location(0)` 標記引數。您也將格式定義為 `float32x2`，也就是 2D 向量，因此在 WGSL 中，您的引數是 `vec2f`。您可以視需要命名，但由於這些代表頂點位置，因此 `pos` 等名稱似乎很自然。

4. 將著色器函式變更為下列程式碼：

index.html (createShaderModule 程式碼)

```
@vertex
fn vertexMain(@location(0) pos: vec2f) ->
    @builtin(position) vec4f {
        return vec4f(0, 0, 0, 1);
    }
```

現在您需要傳回該位置。由於位置是 2D 向量，而傳回類型是 4D 向量，因此您必須稍微變更位置。您要做的就是從位置引數中取出兩個元件，並將其放在傳回向量的前兩個元件中，最後兩個元件則分別保留為 0 和 1。

5. 明確指出要使用的位置元件，傳回正確位置：

index.html (createShaderModule 程式碼)

```
@vertex
fn vertexMain(@location(0) pos: vec2f) ->
    @builtin(position) vec4f {
        return vec4f(pos.x, pos.y, 0, 1);
    }
```

不過，由於這類對應在著色器中非常常見，您也可以將位置向量做為第一個引數傳入簡便的簡寫，這表示相同的事物。

6. 使用下列程式碼重新編寫 `return` 陳述式：

index.html (createShaderModule 程式碼)

```
@vertex
fn vertexMain(@location(0) pos: vec2f) ->
    @builtin(position) vec4f {
        return vec4f(pos, 0, 1);
    }
```

這就是您的初始頂點著色器！這非常簡單，只要有效傳遞位置即可，但這已足夠開始使用。

定義片段著色器

接下來是片段著色器。片段著色器與頂點著色器的運作方式非常相似，但片段著色器是針對繪製的每個像素叫用，而不是每個頂點。

片段著色器一律會在頂點著色器之後呼叫。GPU 會接收頂點著色器的輸出內容並三角剖分，從三點一組的點集建立三角形。接著，系統會點陣化每個三角形，找出輸出顏色附件中包含的像素，然後針對每個像素呼叫片段著色器一次。片段著色器會傳回顏色，通常是根據頂點著色器傳送的值和紋理等素材計算而來，GPU 會將這些顏色寫入顏色附件。

與頂點著色器相同，片段著色器會以大規模平行方式執行。在輸入和輸出方面，片段著色器比頂點著色器更有彈性，但您可以將片段著色器視為單純為每個三角形的每個像素傳回一種顏色。

WGSL 片段著色器函式會以 `@fragment` 屬性標示，並傳回 `vec4f`。不過，在這種情況下，向量代表的是顏色，而非位置。傳回值必須具備 `@location` 屬性，才能指出傳回的顏色要寫入 `beginRenderPass` 呼叫中的哪個 `colorAttachment`。由於只有一個附件，因此位置為 0。

1. 建立空白的 `@fragment` 函式，如下所示：

index.html (createShaderModule 程式碼)

```
@fragment
fn fragmentMain() -> @location(0) vec4f {

}
```

傳回向量的四個元件分別是紅色、綠色、藍色和 Alpha 色彩值，解讀方式與您先前在 `beginRenderPass` 中設定的 `clearValue` 完全相同。因此 `vec4f(1, 0, 0, 1)` 是亮紅色，看起來很適合您的方塊。不過，你可以隨意設定顏色！

2. 設定傳回的顏色向量，如下所示：

index.html (createShaderModule 程式碼)

```
@fragment
fn fragmentMain() -> @location(0) vec4f {
    return vec4f(1, 0, 0, 1); // (Red, Green, Blue, Alpha)
}
```

這就是完整的片段著色器！這並不是非常有趣的著色器，它只會將每個三角形的每個像素設為紅色，但目前這樣就夠了。

再次提醒，新增上述著色器程式碼後，您的 `createShaderModule` 呼叫現在應如下所示：

index.html

```
const cellShaderModule = device.createShaderModule({
    label: 'Cell shader',
    code: `
        @vertex
        fn vertexMain(@location(0) pos: vec2f) ->
            @builtin(position) vec4f {
            return vec4f(pos, 0, 1);
        }

        @fragment
        fn fragmentMain() -> @location(0) vec4f {
            return vec4f(1, 0, 0, 1);
        }
    `
});
```

附註：您也可以視需要為頂點和片段著色器建立個別的著色器模組。舉例來說，如果您想搭配同一個頂點著色器使用多個不同的片段著色器，這項功能就非常實用。

建立算繪管道

著色器模組無法單獨用於算繪。您必須將其做為 `device.createRenderPipeline()` 呼叫所建立的 `GPURenderPipeline` 一部分。算繪管道會控管幾何圖形的繪製方式，包括使用的著色器、如何解讀頂點緩衝區中的資料，以及要算繪的幾何圖形類型 (線條、點、三角形等)。

算繪管道是整個 API 中最複雜的物件，但請放心！您可以傳遞給這個函式的大部分值都是選用值，您只需要提供幾個值即可開始。

- 建立算繪管道，如下所示：

index.html

```
const cellPipeline = device.createRenderPipeline({
  label: "Cell pipeline",
  layout: "auto",
  vertex: {
    module: cellShaderModule,
    entryPoint: "vertexMain",
    buffers: [vertexBufferLayout]
  },
  fragment: {
    module: cellShaderModule,
    entryPoint: "fragmentMain",
    targets: [{
      format: canvasFormat
    }]
  }
});
```

每個管道都需要 `layout`，說明管道需要的輸入類型 (頂點緩衝區除外)，但您實際上沒有任何輸入。幸好，您現在可以傳遞 `"auto"`，管道會從著色器建構自己的版面配置。

接著，請提供 `vertex` 階段的詳細資料。`module` 是包含頂點著色器的 `GPUShaderModule`，而 `entryPoint` 則提供著色器程式碼中函式的名稱，該函式會針對每個頂點叫用呼叫。(單一著色器模組中可以有多個 `@vertex` 和 `@fragment` 函式！)緩衝區是 `GPUVertexBufferLayout` 物件的陣列，說明資料在您使用此管道的頂點緩衝區中封裝的方式。幸好您先前已在 `vertexBufferLayout` 中定義這項功能！您可以在這裡傳入。

最後，您會看到 `fragment` 階段的詳細資料。這也包括著色器 *模組*和 `entryPoint`，例如頂點階段。最後一項工作是定義這個管道使用的 `targets`。這是字典陣列，提供管道輸出至色彩附件的詳細資料，例如紋理 `format`。這些詳細資料必須與這個管道使用的任何算繪通道中提供的紋理相符。`colorAttachments` 算繪通道會使用畫布內容中的紋理，並使用您在 `canvasFormat` 中儲存的值做為格式，因此您在此傳遞的格式相同。

這還遠遠不及建立算繪管道時可指定的所有選項，但已足夠滿足本程式碼研究室的需求！

注意：為什麼這個物件要封裝這麼多東西？這有幾個原因。首先，並非所有選項組合都有效，因此只要在一個位置提供所有選項，您就能在建立時輕鬆判斷管道是否可用。這樣一來，您就不必檢查那麼多項目，因此之後使用管道繪圖時會更快。(這與 WebGL 大相逕庭，因為 WebGL 必須在每次繪製呼叫時驗證大量設定)。

第二個原因是，在繪製時，您只需單一 JavaScript 呼叫，就能將大量資訊傳送至算繪通道。這樣一來，您整體需要發出的呼叫次數就會減少，算繪作業也會更有效率。(同樣地，這也是 WebGL 的弱項)。

繪製正方形

這樣一來，您就擁有繪製正方形所需的一切！

1. 如要繪製正方形，請返回 `encoder.beginRenderPass()` 和 `pass.end()` 這對呼叫，然後在兩者之間新增下列指令：

index.html

```
// After encoder.beginRenderPass()

pass.setPipeline(cellPipeline);
pass.setVertexBuffer(0, vertexBuffer);
pass.draw(vertices.length / 2); // 6 vertices

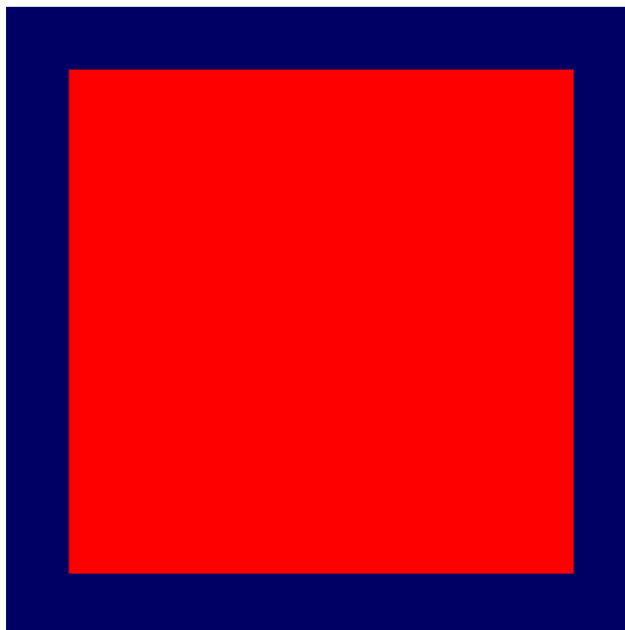
// before pass.end()
```

這會提供 WebGPU 繪製正方形所需的所有資訊。首先，您可以使用 `setPipeline()` 指出要用於繪圖的管道。包括使用的著色器、頂點資料的版面配置和其他相關狀態資料。

接著，您會使用包含正方形頂點的緩衝區呼叫 `setVertexBuffer()`。您使用 `0` 呼叫這個緩衝區，因為這個緩衝區對應於目前管道 `vertex.buffers` 定義中的第 0 個元素。

最後，您會呼叫 `draw()`，在完成所有設定後，這個呼叫似乎簡單得令人意外。您只需要傳遞應算繪的頂點數量，系統會從目前設定的頂點緩衝區中提取，並使用目前設定的管道解讀。您可以將其硬式編碼為 `6`，但從頂點陣列計算 (每個頂點 12 個浮點值 / 2 個座標 == 6 個頂點) 表示，如果您決定將正方形替換為圓形等形狀，需要手動更新的內容較少。

2. 重新整理畫面，(終於) 看到所有努力的成果：一個大大的彩色正方形。



5. 繪製格線

首先，恭喜你！使用大多數 GPU API 時，在畫面上取得第一批幾何圖形通常是最困難的步驟之一。您可以在這裡將所有操作分成較小的步驟，方便隨時驗證進度。

本節的學習內容如下：

- 如何從 JavaScript 將變數 (稱為統一變數) 傳遞至著色器。
- 如何使用制服變更算繪行為。
- 如何使用例項繪製同一幾何體的許多不同變體。

定義格線

如要算繪格線，您需要瞭解格線的基本資訊。包含多少個儲存格 (寬度和高度)？這取決於開發人員，但為了簡化作業，請將格線視為正方形 (寬度和高度相同)，並使用 2 的乘方大小。(這樣之後的數學運算會比較簡單)。您最終想將格線放大，但為了方便說明本節使用的部分數學概念，請在本節的其餘部分將格線大小設為 4x4。之後再擴大！

- 在 JavaScript 程式碼頂端新增常數，定義格線大小。

index.html

```
const GRID_SIZE = 4;
```

接著，您需要更新正方形的算繪方式，以便在畫布上放置 `GRID_SIZE` x `GRID_SIZE` 個正方形。這表示正方形必須小很多，而且數量要非常多。

現在，其中一種做法是大幅擴大頂點緩衝區，並在其中定義 `GRID_SIZE` 次數的 `GRID_SIZE` 個正方形，且大小和位置都正確。事實上，這段程式碼還不錯！只要幾個 for 迴圈和一點數學知識，但這樣也無法充分運用 GPU，且會使用比必要更多的記憶體來達成效果。本節將介紹更適合 GPU 的做法。

建立統一緩衝區

首先，您需要將所選格線大小傳達給著色器，因為著色器會使用該大小變更顯示內容的方式。您可以將大小硬式編碼到著色器中，但這表示每次想變更格線大小時，都必須重新建立著色器和算繪管道，這會耗費大量資源。更好的做法是將格線大小提供給著色器做為制服。

您稍早瞭解到，系統會將頂點緩衝區中的不同值傳遞至頂點著色器的每次叫用。Uniform 是緩衝區中的值，每次呼叫時都相同。這類屬性可用於傳達幾何圖形 (例如位置)、完整動畫影格 (例如目前時間)，甚至是整個應用程式生命週期 (例如使用者偏好設定) 的常見值。

- 新增下列程式碼，建立統一緩衝區：

index.html


```
// Create a uniform buffer that describes the grid.
const uniformArray = new Float32Array([GRID_SIZE, GRID_SIZE]);
const uniformBuffer = device.createBuffer({
  label: "Grid Uniforms",
  size: uniformArray.byteLength,
  usage: GPUBufferUsage.UNIFORM | GPUBufferUsage.COPY_DST,
});
device.queue.writeBuffer(uniformBuffer, 0, uniformArray);
```

這應該看起來很眼熟，因為這幾乎與您先前用於建立頂點緩衝區的程式碼完全相同！這是因為系統會透過與頂點相同的 `GPUBuffer` 物件，將制服傳達至 WebGPU API，主要差異在於這次的 `usage` 包含 `GPUBufferUsage.UNIFORM`，而非 `GPUBufferUsage.VERTEX`。

注意：您也可以在此使用 `Uint32Array`，因為格線大小值不是分數，但在這個特定情況下，您還是會在多個位置將值轉換為浮點值，因此會讓即將進行的著色器程式碼變更變得有點混亂。請注意，浮點數並非唯一可放在 `GPUBuffer` 中的資料類型！

在著色器中存取制服

- 新增下列程式碼來定義制服：

index.html (createShaderModule 呼叫)

```
// At the top of the `code` string in the createShaderModule() call
@group(0) @binding(0) var<uniform> grid: vec2f;

@vertex
fn vertexMain(@location(0) pos: vec2f) ->
  @builtin(position) vec4f {
  return vec4f(pos / grid, 0, 1);
}

// ...fragmentMain is unchanged
```

這會在著色器中定義名為 `grid` 的統一值，也就是與您剛複製到統一緩衝區的陣列相符的 2D 浮點向量。同時指定制服的繫結位置為 `@group(0)` 和 `@binding(0)`。稍後會說明這些值的意義。

然後，您可以在著色器程式碼的其他位置，視需要使用格線向量。在這段程式碼中，您會將頂點位置除以格線向量。由於 `pos` 是 2D 向量，`grid` 也是 2D 向量，因此 WGSL 會執行元件式除法。換句話說，結果與 `vec2f(pos.x / grid.x, pos.y / grid.y)` 相同。

這類向量運算在 GPU 著色器中非常常見，因為許多算繪和運算技術都依賴這類運算！

以您的情況來說，這表示 (如果您使用格線大小 4)，您算繪的正方形會是原始大小的四分之一。如果想在一系列或一行中放入四個，這個尺寸就非常適合！

建立繫結群組

不過，在著色器中宣告制服不會將其與您建立的緩衝區連結。如要這麼做，請建立並設定繫結群組。

繫結群組是您要同時提供給著色器的資源集合。這可能包括多種緩衝區，例如統一緩衝區，以及紋理和取樣器等其他資源 (這裡未涵蓋，但這些是 WebGPU 算繪技術的常見部分)。

- 在建立統一緩衝區和算繪管道後，加入下列程式碼，即可使用統一緩衝區建立繫結群組：

index.html

```
const bindGroup = device.createBindGroup({
  label: "Cell renderer bind group",
  layout: cellPipeline.getBindGroupLayout(0),
  entries: [{
    binding: 0,
    resource: { buffer: uniformBuffer }
  }],
});
```

除了現在的標準 `label`，您還需要 `layout`，說明這個繫結群組包含哪些類型的資源。您會在後續步驟中深入瞭解這項內容，但目前可以放心地向管道要求繫結群組版面配置，因為您是使用 `layout: "auto"` 建立管道。這會導致管道從您在著色器程式碼本身中宣告的繫結，自動建立繫結群組版面配置。在本例中，您要求它 `getBindGroupLayout(0)`，其中 `0` 對應於您在著色器中輸入的 `@group(0)`。

指定版面配置後，請提供 `entries` 陣列。每個項目都是字典，至少包含下列值：

- **binding**，這會對應您在著色器中輸入的 `@binding()` 值。在本例中為 `0`。
- **resource**，這是您要公開給指定繫結索引處變數的實際資源。在本例中，就是統一緩衝區。

此函式會傳回 `GPUBindGroup`，這是不透明的不可變動控制代碼。建立繫結群組後，您就無法變更該群組指向的資源，但可以變更這些資源的內容。舉例來說，如果您變更統一緩衝區，使其包含新的格線大小，則使用這個繫結群組的後續繪圖呼叫會反映這項變更。

繫結繫結群組

建立繫結群組後，您仍需告知 WebGPU 在繪製時使用該群組。幸好這項作業相當簡單。

1. 返回算繪通道，在 `draw()` 方法之前新增這一行：

index.html

```
pass.setPipeline(cellPipeline);
pass.setVertexBuffer(0, vertexBuffer);

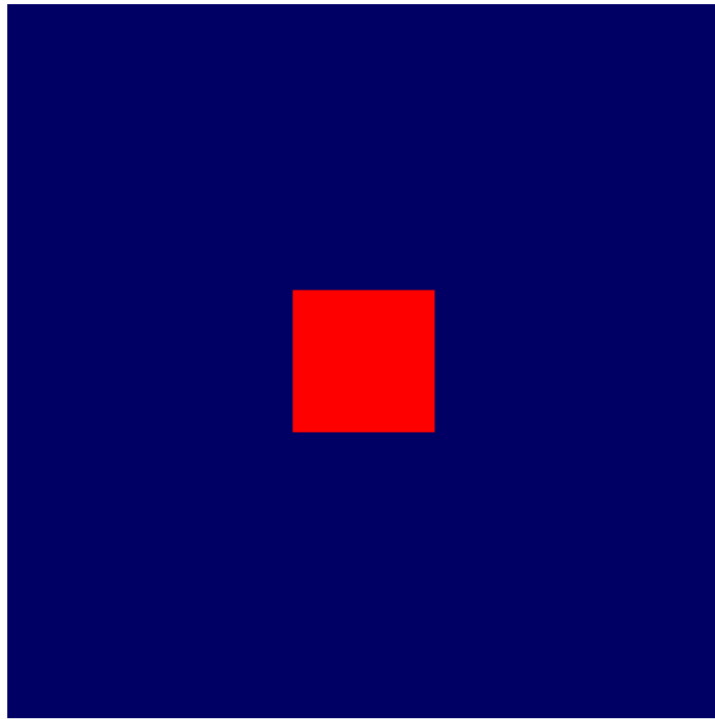
pass.setBindGroup(0, bindGroup); // New line!

pass.draw(vertices.length / 2);
```

做為第一個引數傳遞的 `0` 對應於著色器程式碼中的 `@group(0)`。您表示屬於 `@group(0)` 的每個 `@binding` 都會使用這個繫結群組中的資源。

現在，統一緩衝區會公開給著色器！

2. 重新整理頁面後，您應該會看到類似下方的畫面：

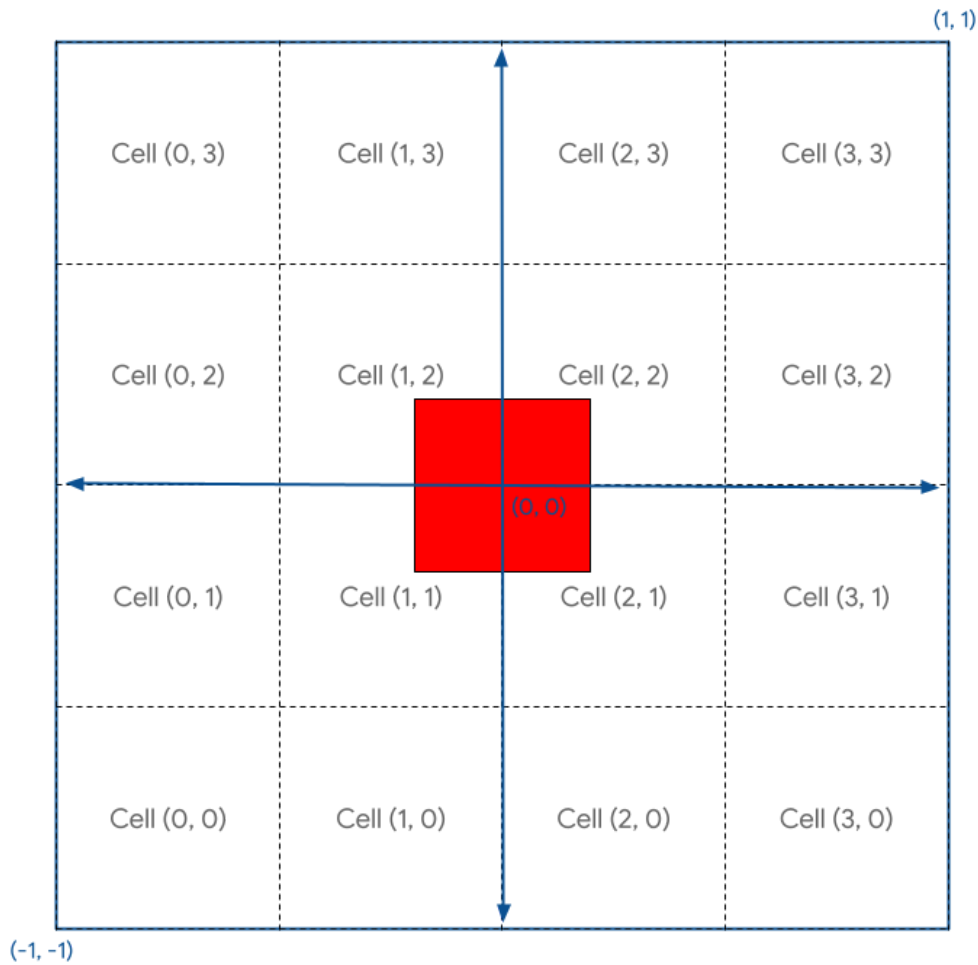


太棒了！現在正方形的大小只有原本的四分之一！雖然不多，但這表示您的制服已實際套用，且著色器現在可以存取格線大小。

在著色器中操控幾何圖形

現在您可以在著色器中參照格線大小，開始進行一些工作，以操控要算繪的幾何圖形，配合所需的格線模式。如要達成這個目標，請仔細思考您想達成的確切目標。

您需要從概念上將畫布劃分成個別儲存格。為了維持慣例，也就是 X 軸值會隨著向右移動而增加，Y 軸值會隨著向上移動而增加，假設第一個儲存格位於畫布的左下角。這樣一來，版面配置就會如下所示，目前的方形幾何圖形位於中間：



您的挑戰是找出著色器中的方法，根據儲存格座標將正方形幾何體放置在任一儲存格中。

首先，您會發現正方形並未與任何儲存格對齊，因為正方形定義為環繞畫布中心。您會希望將正方形移動半個儲存格，以便在儲存格內整齊排列。

其中一種修正方法是更新正方形的頂點緩衝區。舉例來說，如果將頂點從 (-0.8, -0.8) 移至 (0.1, 0.1)，即可將這個正方形移至更符合儲存格邊界的對齊位置。不過，由於您可以完全掌控著色器中頂點的處理方式，因此使用著色器程式碼將頂點推入適當位置，同樣簡單容易！

1. 使用下列程式碼修改頂點著色器模組：

index.html (createShaderModule 呼叫)

```
@group(0) @binding(0) var<uniform> grid: vec2f;

@vertex
fn vertexMain(@location(0) pos: vec2f) ->
    @builtin(position) vec4f {

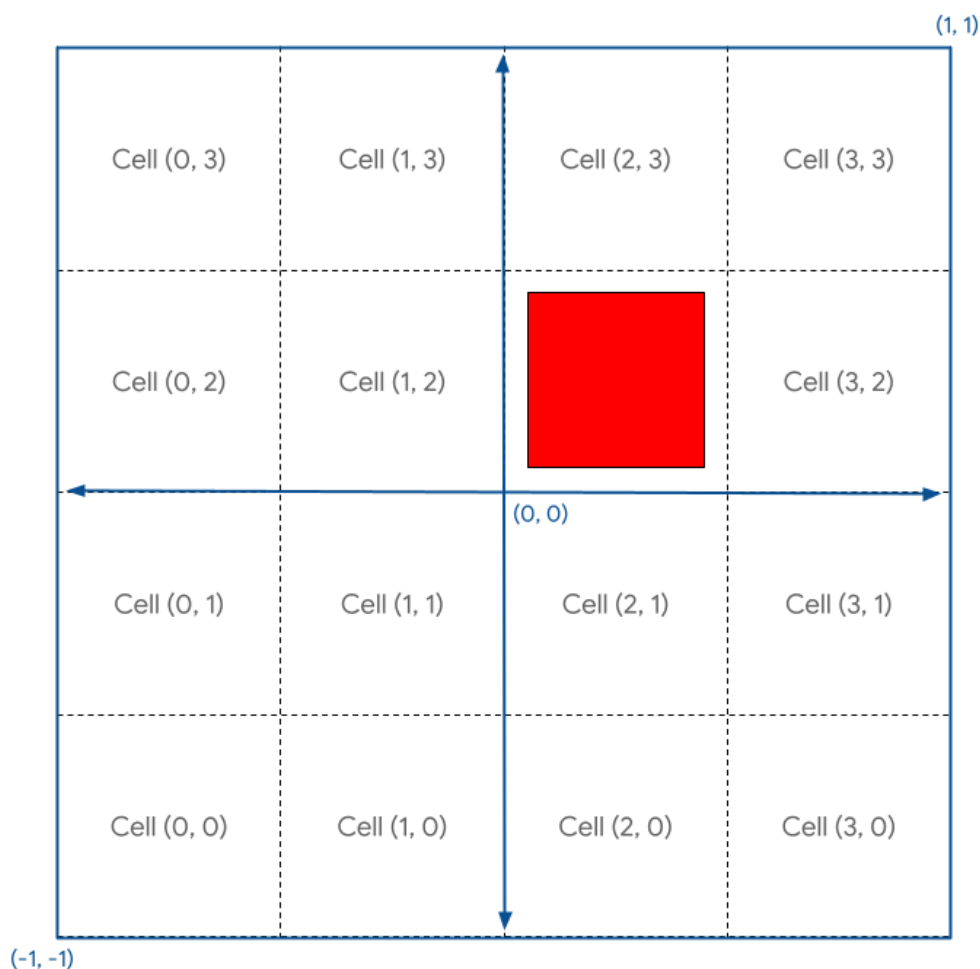
    // Add 1 to the position before dividing by the grid size.
    let gridPos = (pos + 1) / grid;

    return vec4f(gridPos, 0, 1);
}
```

注意：為方便閱讀，位置計算會分成自己的變數。請注意，這是使用 `let` 宣告，但請注意，WGSL 中的 `let` 與 JavaScript 中的 `let` 意義不同！在 WGSL 中，`let` 的行為比較類似於 JavaScript 的 `const`，表示變數的值在指派後不會變更。如果確實需要在 WGSL 中宣告變數後變更變數，請改用 `var`。

附註：由於 `pos` 是這裡的 `vec2f`，因此新增 `1` 會逐一新增元件。這與說 `pos + vec2f(1, 1)` 相同。加法、乘法和除法也適用相同做法！

這會將每個頂點向上和向右移動一個單位 (請注意，這是剪輯空間的一半)，然後再除以格線大小。結果是與原點稍微錯開的方塊，且與格線對齊。



接著，由於畫布的座標系統會將 (0, 0) 放在中心，並將 (-1, -1) 放在左下角，而您希望 (0, 0) 位於左下角，因此您需要在除以格線大小後，將幾何圖形的位置平移 (-1, -1)，以便將其移至該角落。

2. 翻譯幾何圖形的位置，如下所示：

index.html (createShaderModule 呼叫)

```

@group(0) @binding(0) var<uniform> grid: vec2f;

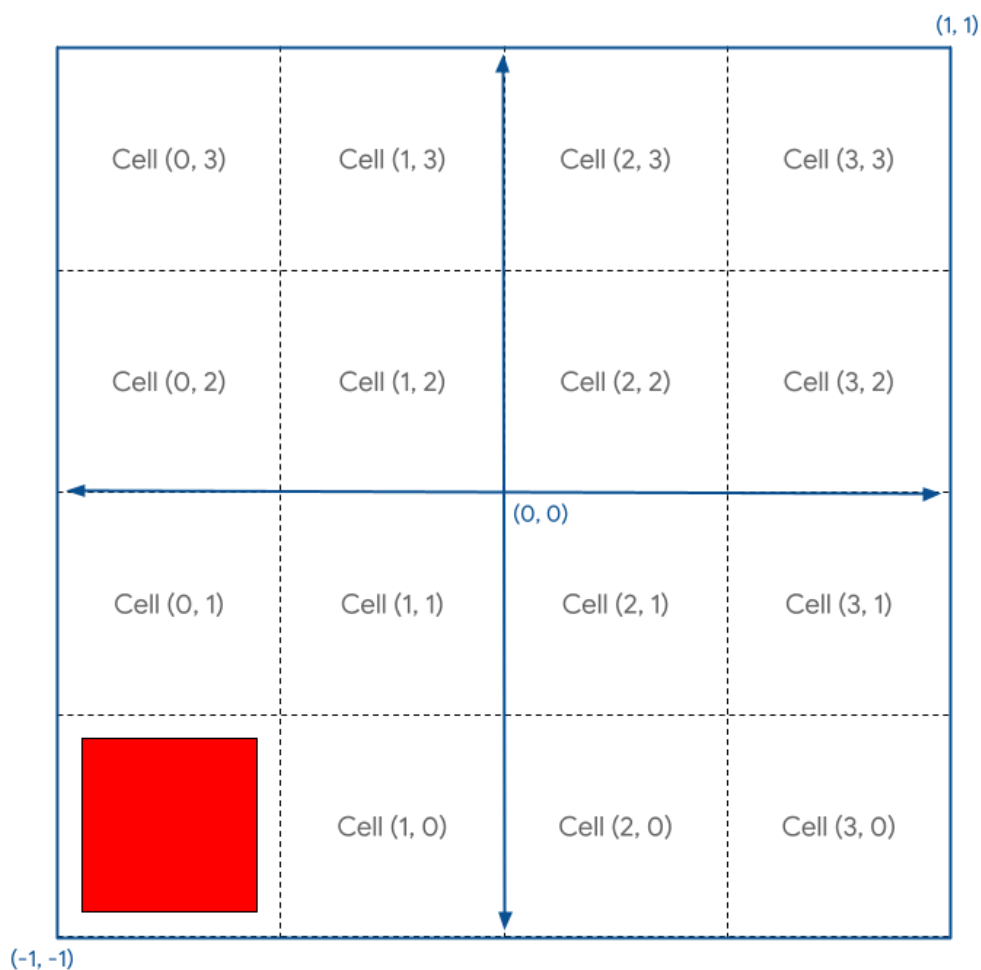
@vertex
fn vertexMain(@location(0) pos: vec2f) ->
    @builtin(position) vec4f {

    // Subtract 1 after dividing by the grid size.
    let gridPos = (pos + 1) / grid - 1;

    return vec4f(gridPos, 0, 1);
}

```

現在，正方形已順利放置在儲存格 (0, 0) 中！



如果想將圖片放在其他儲存格，如要找出這個值，請在著色器中宣告 `cell` 向量，並填入 `let cell = vec2f(1, 1)` 等靜態值。

如果您將該項目新增至 `gridPos`，演算法中的 `- 1` 就會復原，因此這並非您要的結果。您希望每個儲存格只移動一個格線單位 (畫布的四分之一)。看來你還需要再除以 `grid` ！

3. 變更格線位置，如下所示：

index.html (createShaderModule 呼叫)

```

@group(0) @binding(0) var<uniform> grid: vec2f;

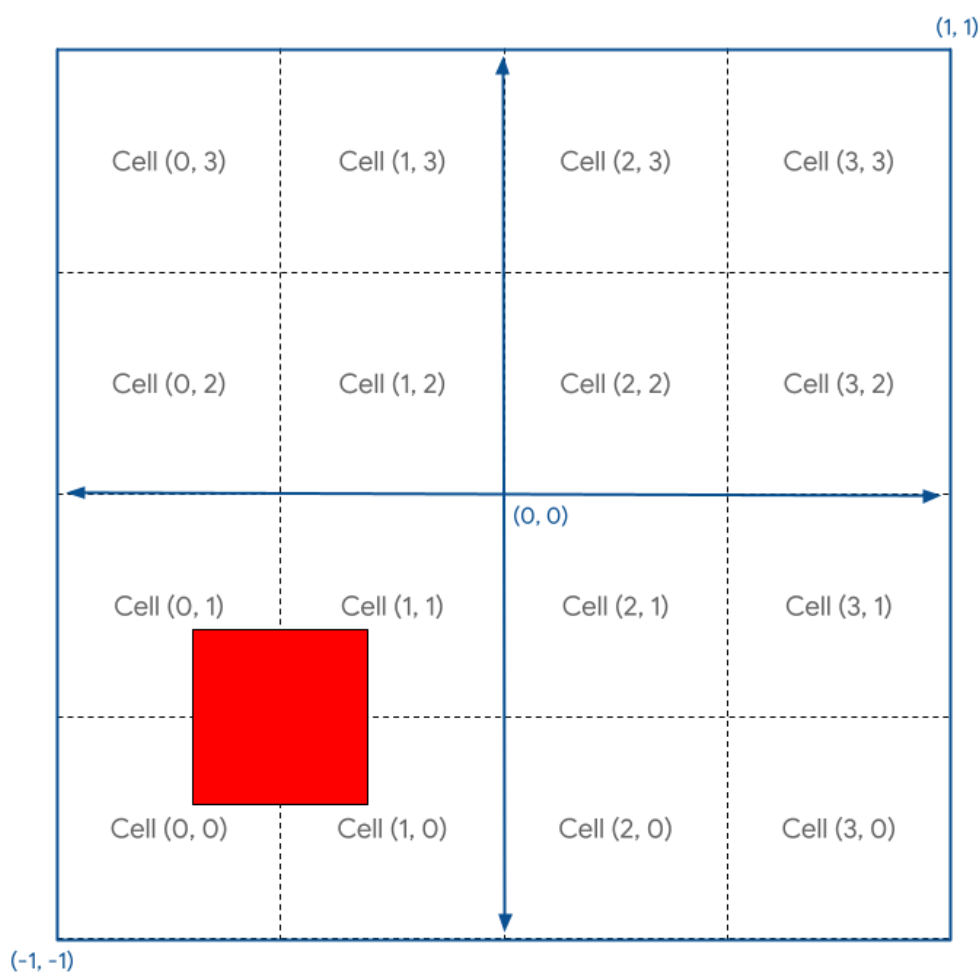
@vertex
fn vertexMain(@location(0) pos: vec2f) ->
    @builtin(position) vec4f {

    let cell = vec2f(1, 1); // Cell(1,1) in the image above
    let cellOffset = cell / grid; // Compute the offset to cell
    let gridPos = (pos + 1) / grid - 1 + cellOffset; // Add it here!

    return vec4f(gridPos, 0, 1);
}

```

現在重新整理，您會看到以下內容：



嗯...不符合需求。

這是因為畫布座標的範圍是 -1 到 +1，實際跨越 2 個單位。也就是說，如要將頂點移動四分之一的畫布，必須移動 0.5 個單位。使用 GPU 座標時，很容易發生這類錯誤！所幸修正方式同樣簡單。

4. 將位移量乘以 2，如下所示：

index.html (createShaderModule 呼叫)

```

@group(0) @binding(0) var<uniform> grid: vec2f;

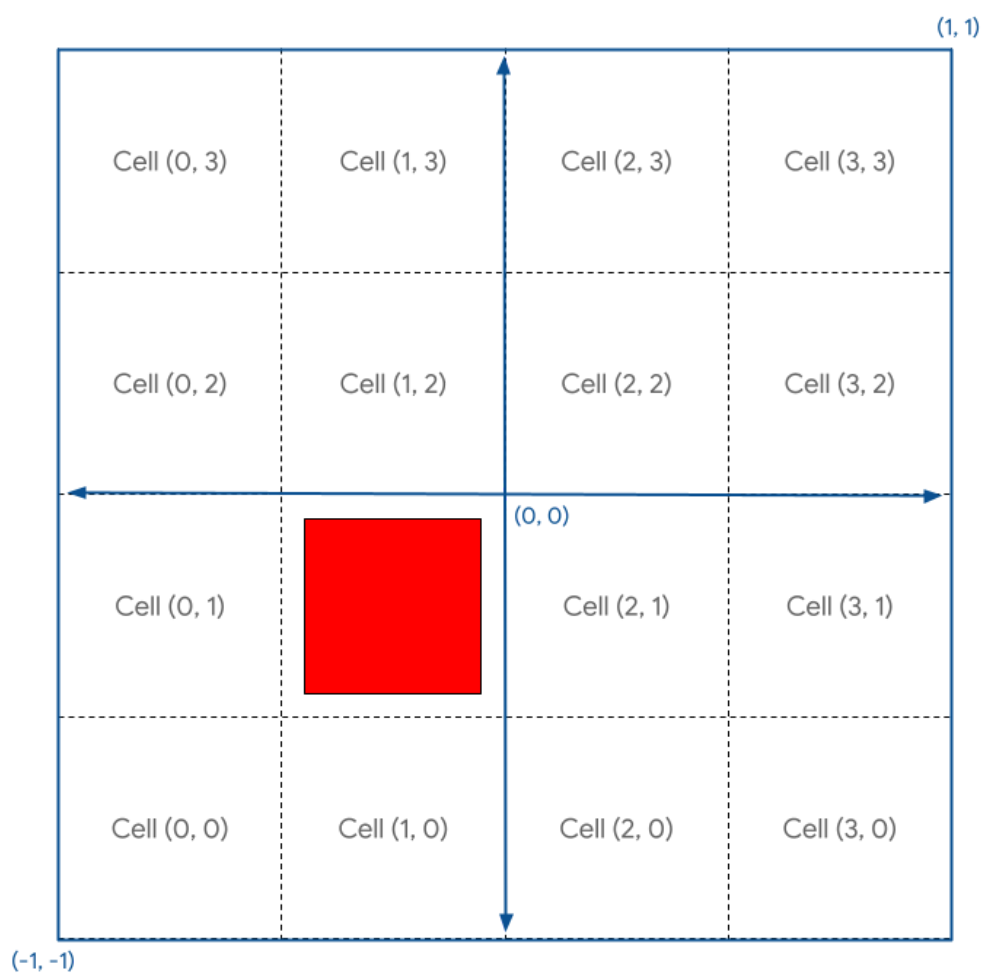
@vertex
fn vertexMain(@location(0) pos: vec2f) ->
    @builtin(position) vec4f {

    let cell = vec2f(1, 1);
    let cellOffset = cell / grid * 2; // Updated
    let gridPos = (pos + 1) / grid - 1 + cellOffset;

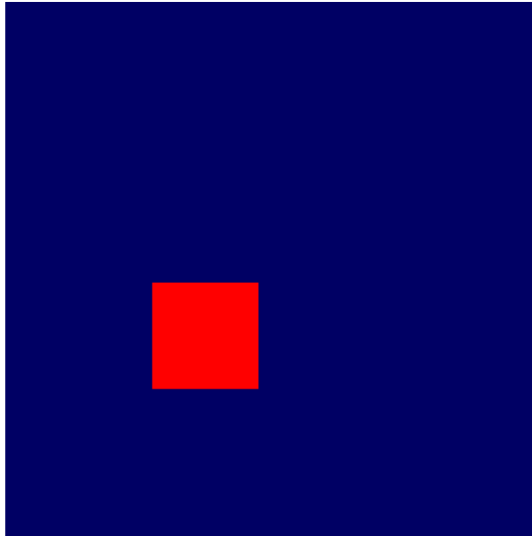
    return vec4f(gridPos, 0, 1);
}

```

這樣就能獲得所需資訊。



螢幕截圖如下所示：



此外，您現在可以將 `cell` 設為格線範圍內的任何值，然後重新整理，即可在所需位置看到正方形。

繪製執行個體

現在您已可透過一些數學運算，將正方形放置在所需位置，下一步是在格線的每個儲存格中算繪一個正方形。

其中一種做法是將儲存格座標寫入統一緩衝區，然後針對格線中的每個方塊呼叫一次 `draw`，每次都更新統一緩衝區。不過，這樣做會非常緩慢，因為 GPU 每次都必須等待 JavaScript 寫入新的座標。如要讓 GPU 發揮良好效能，其中一個關鍵就是盡量減少等待系統其他部分的時間！

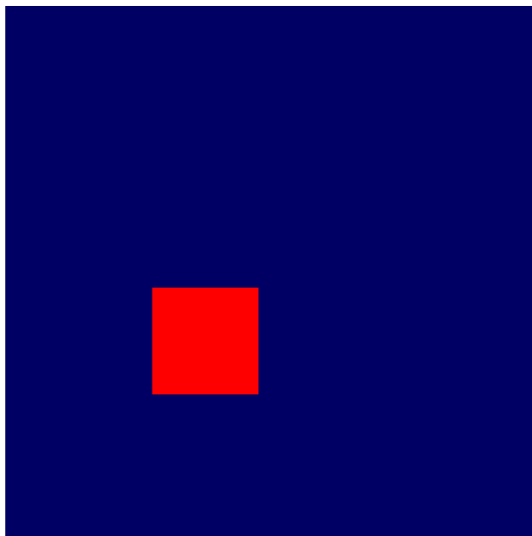
不過，您可以改用例項化技術。例項化可讓您透過單一 `draw` 呼叫，指示 GPU 繪製相同幾何圖形的多個副本，這比為每個副本呼叫一次 `draw` 快得多。每個幾何體副本都稱為「執行個體」。

1. 如要告知 GPU，您希望有足夠的方形執行個體填滿格線，請在現有的繪製呼叫中新增一個引數：

index.html

```
pass.draw(vertices.length / 2, GRID_SIZE * GRID_SIZE);
```

這會告知系統，您希望繪製正方形的六個 (`vertices.length / 2`) 頂點 16 (`GRID_SIZE * GRID_SIZE`) 次。但如果重新整理頁面，仍會看到下列訊息：



這是因為這是因為您在同一個位置繪製了所有 16 個正方形。您需要在著色器中加入一些額外邏輯，根據每個執行個體重新定位幾何。

在著色器中，除了來自頂點緩衝區的 `pos` 等頂點屬性，您也可以存取所謂的 WGSL [內建值](#)。這些是 WebGPU 計算的值，其中一個值是 `instance_index`。`instance_index` 是從 0 到 `number of instances - 1` 的不帶正負號 32 位元數字，可用於著色器邏輯。對於屬於相同執行個體的每個處理頂點，這個值都相同。也就是說，系統會使用 0 的 `instance_index` 呼叫頂點著色器六次，頂點緩衝區中的每個位置各呼叫一次。然後再重複六次，每次都使用 `instance_index` 的 1，接著再重複六次，每次都使用 `instance_index` 的 2，依此類推。

如要查看實際效果，您必須將 `instance_index` 內建函式新增至著色器輸入內容。做法與位置相同，但請使用 `@builtin(instance_index)`，而非以 `@location` 屬性標記，然後為引數命名。(您可以呼叫 `instance` 來比對範例程式碼)。然後將其做為著色器邏輯的一部分！

2. 使用 `instance` 取代儲存格座標：

index.html

```
@group(0) @binding(0) var<uniform> grid: vec2f;

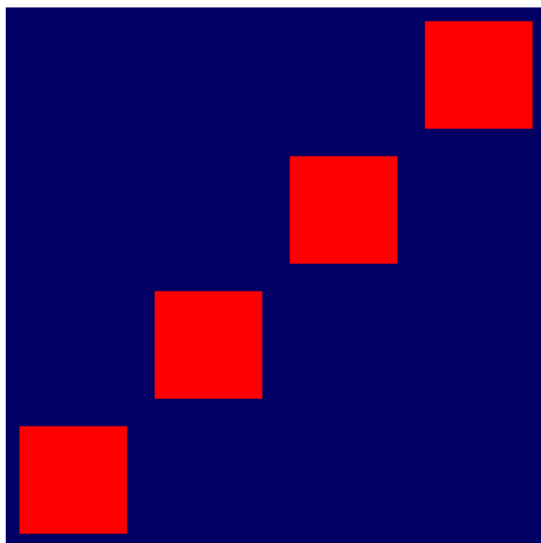
@vertex
fn vertexMain(@location(0) pos: vec2f,
               @builtin(instance_index) instance: u32) ->
    @builtin(position) vec4f {

    let i = f32(instance); // Save the instance_index as a float
    let cell = vec2f(i, i); // Updated
    let cellOffset = cell / grid * 2;
    let gridPos = (pos + 1) / grid - 1 + cellOffset;

    return vec4f(gridPos, 0, 1);
}
```

注意：對於熟悉 JavaScript 但不熟悉其他強型別語言的開發人員來說，WGSL 的強型別可能會造成困擾。`instance_index` 是 `u32` 或不帶正負號的 32 位元整數。但您希望 `cell` 是浮點值向量，因為這是位置數學的運算方式。因此，建立 `cell` 向量時，您必須將 `instance` 包裝在 `f32()` 函式中，明確告知著色器將 `instance` 視為浮點值。這就是所謂的「轉換」型別。

現在重新整理，您會發現確實有多個正方形！但你無法看到全部 16 個。



這是因為您產生的儲存格座標為 (0, 0)、(1, 1)、(2, 2)... 一直到 (15, 15)，但只有前四個座標符合畫布大小。如要建立所需格線，您必須轉換 `instance_index`，讓每個索引對應格線中的不重複儲存格，如下所示：

Instance 12 Cell (0, 3)	Instance 13 Cell (1, 3)	Instance 14 Cell (2, 3)	Instance 15 Cell (3, 3)
Instance 8 Cell (0, 2)	Instance 9 Cell (1, 2)	Instance 10 Cell (2, 2)	Instance 11 Cell (3, 2)
Instance 4 Cell (0, 1)	Instance 5 Cell (1, 1)	Instance 6 Cell (2, 1)	Instance 7 Cell (3, 1)
Instance 0 Cell (0, 0)	Instance 1 Cell (1, 0)	Instance 2 Cell (2, 0)	Instance 3 Cell (3, 0)

這項計算相當簡單。針對每個儲存格的 X 值，您需要 `instance_index` 和格線寬度的模數，這可以使用 `%` 運算子在 WGSL 中執行。並將每個儲存格的 Y 值除以格線寬度，捨棄任何分數餘數。 `instance_index` 您可以使用 WGSL 的 `floor()` 函式執行這項操作。

3. 變更計算方式，如下所示：

index.html

```
@group(0) @binding(0) var<uniform> grid: vec2f;

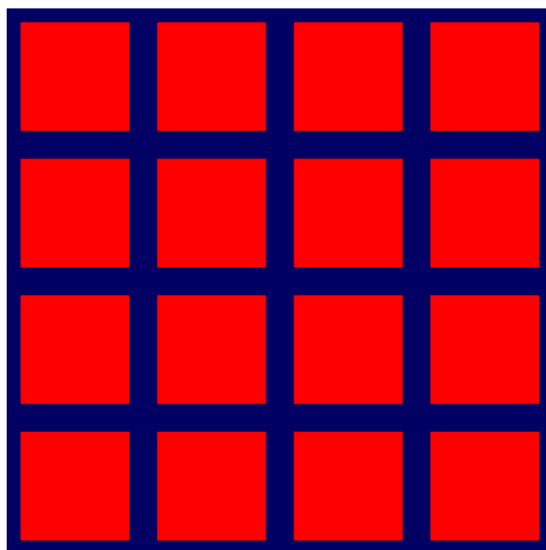
@vertex
fn vertexMain(@location(0) pos: vec2f,
               @builtin(instance_index) instance: u32) ->
    @builtin(position) vec4f {

    let i = f32(instance);
    // Compute the cell coordinate from the instance_index
    let cell = vec2f(i % grid.x, floor(i / grid.x));

    let cellOffset = cell / grid * 2;
    let gridPos = (pos + 1) / grid - 1 + cellOffset;

    return vec4f(gridPos, 0, 1);
}
```

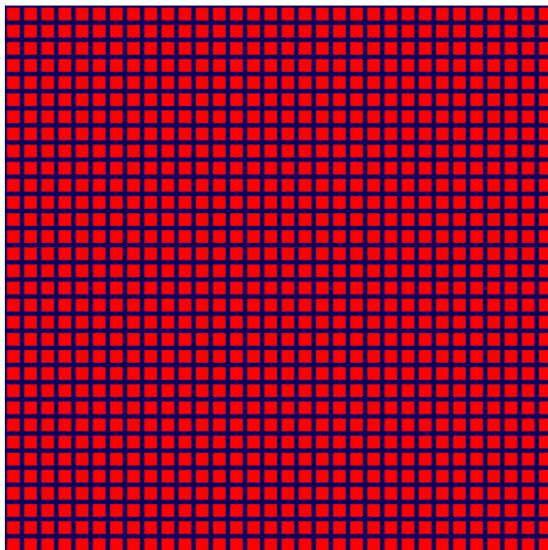
更新程式碼後，您就能看到期待已久的方格網格！



4. 現在可以正常運作了，請返回並調高格線大小！

index.html

```
const GRID_SIZE = 32;
```



大功告成！現在您可以將這個格線做得很大，平均 GPU 都能順利處理。在遇到任何 GPU 效能瓶頸之前，您就會停止看到個別方塊。

注意：您可能會想：「這一切都可以用巢狀迴圈中的一行 Canvas 2D 程式碼完成！」而且... 沒錯！有可能！如果這就是您感興趣的算繪層級，建議您使用 Canvas 2D 等較簡單的 API。本程式碼實驗室著重於介紹 WebGPU 的核心概念，因此不會深入探討 API 的算繪功能。

值得注意的是，如果您想使用 Canvas 2D 滿足這類較簡單的算繪需求，但仍想運用 WebGPU 的運算功能，可以合作使用這些 API！如果所有作業都在 WebGPU 中完成，則需要更多資料副本。如需如何搭配使用 WebGPU 和 Canvas 2D 的範例，請參閱「[WebGPU - All of the cores, none of the canvas](#)」。

6. 加分題：讓畫面更加繽紛！

此時，您已為本程式碼研究室的其餘部分奠定基礎，因此可以輕鬆跳到下一節。雖然所有方格共用同一種顏色也還算堪用，但這並非令人興奮的設計，不是嗎？幸好，只要多一點數學和著色器程式碼，就能讓畫面更明亮！

在著色器中使用結構體

到目前為止，您已從頂點著色器傳遞一筆資料：轉換後的位置。但實際上，您可以從頂點著色器傳回更多資料，然後在片段著色器中使用這些資料！

如要將資料傳遞出頂點著色器，唯一方法就是傳回資料。頂點著色器一律需要傳回位置，因此如要一併傳回任何其他資料，必須將其放在結構體中。WGSL 中的結構體是具名物件型別，包含一或多個具名屬性。屬性也可以使用 `@builtin` 和 `@location` 等屬性標記。您可以在任何函式外部宣告這些函式，然後視需要傳入及傳出函式例項。舉例來說，假設您目前的頂點著色器如下：

index.html (createShaderModule 呼叫)

```
@group(0) @binding(0) var<uniform> grid: vec2f;

@vertex
fn vertexMain(@location(0) pos: vec2f,
               @builtin(instance_index) instance: u32) ->
    @builtin(position) vec4f {

    let i = f32(instance);
    let cell = vec2f(i % grid.x, floor(i / grid.x));
    let cellOffset = cell / grid * 2;
    let gridPos = (pos + 1) / grid - 1 + cellOffset;

    return vec4f(gridPos, 0, 1);
}
```

- 使用函式輸入和輸出的結構體，以相同方式表示：

index.html (createShaderModule 呼叫)

```

struct VertexInput {
    @location(0) pos: vec2f,
    @builtin(instance_index) instance: u32,
};

struct VertexOutput {
    @builtin(position) pos: vec4f,
};

@group(0) @binding(0) var<uniform> grid: vec2f;

@vertex
fn vertexMain(input: VertexInput) -> VertexOutput {
    let i = f32(input.instance);
    let cell = vec2f(i % grid.x, floor(i / grid.x));
    let cellOffset = cell / grid * 2;
    let gridPos = (input.pos + 1) / grid - 1 + cellOffset;

    var output: VertexOutput;
    output.pos = vec4f(gridPos, 0, 1);
    return output;
}

```

請注意，這需要您使用 `input` 參照輸入位置和執行個體索引，且您傳回的 `struct` 必須先宣告為變數，並設定個別屬性。在這個情況下，兩者差異不大，實際上還會讓著色器函式稍微變長，但隨著著色器變得更加複雜，使用結構體是整理資料的好方法。

在頂點和片段函式之間傳遞資料

再次提醒，您的 `@fragment` 函式盡可能簡單：

index.html (createShaderModule 呼叫)

```

@fragment
fn fragmentMain() -> @location(0) vec4f {
    return vec4f(1, 0, 0, 1);
}

```

您沒有採用任何輸入內容，而是傳遞純色 (紅色) 做為輸出內容。不過，如果著色器更瞭解要著色的幾何圖形，您就可以使用額外資料，讓物件更有趣。舉例來說，如果想根據每個方塊的儲存格座標變更顏色，該怎麼做？`@vertex` 階段知道要算繪哪個儲存格，您只需要將該儲存格傳遞至 `@fragment` 階段即可。

如要在頂點和片段階段之間傳遞任何資料，您必須將資料納入輸出結構體，並選擇 `@location`。由於您要傳遞儲存格座標，請將其新增至先前的 `VertexOutput` 結構體，然後在傳回前於 `@vertex` 函式中設定。

1. 變更頂點著色器的傳回值，如下所示：

index.html (createShaderModule 呼叫)

```

struct VertexInput {
    @location(0) pos: vec2f,
    @builtin(instance_index) instance: u32,
};

struct VertexOutput {
    @builtin(position) pos: vec4f,
    @location(0) cell: vec2f, // New line!
};

@group(0) @binding(0) var<uniform> grid: vec2f;

@vertex
fn vertexMain(input: VertexInput) -> VertexOutput {
    let i = f32(input.instance);
    let cell = vec2f(i % grid.x, floor(i / grid.x));
    let cellOffset = cell / grid * 2;
    let gridPos = (input.pos + 1) / grid - 1 + cellOffset;

    var output: VertexOutput;
    output.pos = vec4f(gridPos, 0, 1);
    output.cell = cell; // New line!
    return output;
}

```

2. 在 `@fragment` 函式中，新增具有相同 `@location` 的引數，即可接收值。(名稱不必相同，但如果相同，追蹤項目會比較容易！)

index.html (createShaderModule 呼叫)

```

@fragment
fn fragmentMain(@location(0) cell: vec2f) -> @location(0) vec4f {
    // Remember, fragment return values are (Red, Green, Blue, Alpha)
    // and since cell is a 2D vector, this is equivalent to:
    // (Red = cell.x, Green = cell.y, Blue = 0, Alpha = 1)
    return vec4f(cell, 0, 1);
}

```

3. 或者，您也可以改用結構體：

index.html (createShaderModule 呼叫)


```

struct FragInput {
    @location(0) cell: vec2f,
};

@fragment
fn fragmentMain(input: FragInput) -> @location(0) vec4f {
    return vec4f(input.cell, 0, 1);
}

```

4. 由於這兩個函式是在同一個著色器模組中定義，因此另一個替代方案是重複使用 `@vertex` 階段的輸出結構體！這樣一來，名稱和位置自然會保持一致，因此傳遞值時會更加輕鬆。

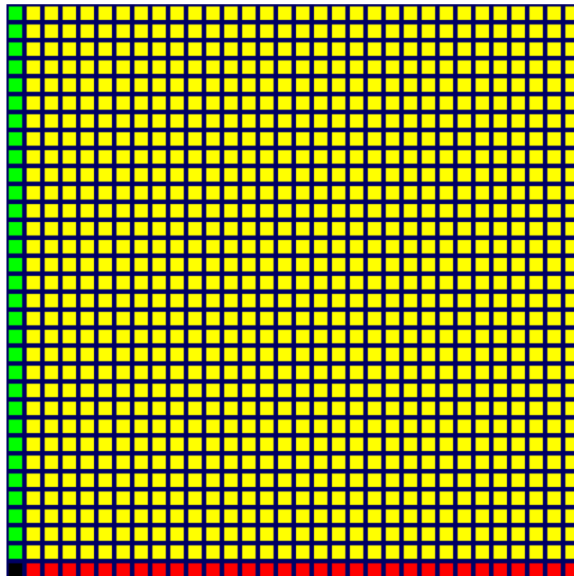
index.html (createShaderModule 呼叫)

```

@fragment
fn fragmentMain(input: VertexOutput) -> @location(0) vec4f {
    return vec4f(input.cell, 0, 1);
}

```

無論選擇哪種模式，結果都是您可以在 `@fragment` 函式中存取儲存格編號，並使用該編號來影響顏色。使用上述任一程式碼，輸出內容如下所示：



現在的顏色確實比較多，但看起來不太美觀。您可能會想知道，為什麼只有左側和底部列不同。這是因為您從 `@fragment` 函式傳回的顏色值會預期每個管道的值介於 0 到 1 之間，超出該範圍的值會限制在該範圍內。另一方面，每個軸的儲存格值範圍為 0 到 32。因此您會看到第一列和第一欄的紅色或綠色色版立即達到 1 的完整值，之後每個儲存格都會固定在相同的值。

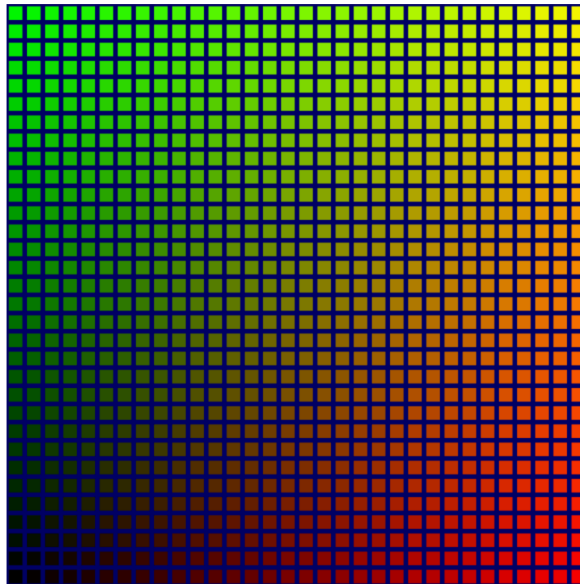
如要讓顏色之間的轉換更平滑，您需要為每個顏色通道傳回分數值，最好是沿著每個軸從零開始，到一結束，這表示還要再除以 `grid`！

5. 變更片段著色器，如下所示：

index.html (createShaderModule 呼叫)

```
@fragment
fn fragmentMain(input: VertexOutput) -> @location(0) vec4f {
    return vec4f(input.cell/grid, 0, 1);
}
```

重新整理頁面，您會發現新程式碼**確實**在整個格線中提供更精美的色彩漸層。



雖然這確實有所改善，但現在左下角出現了令人遺憾的暗角，網格變成黑色。開始模擬生命遊戲時，格線中難以看清的部分會遮蓋畫面，如果能讓畫面更明亮就好了。

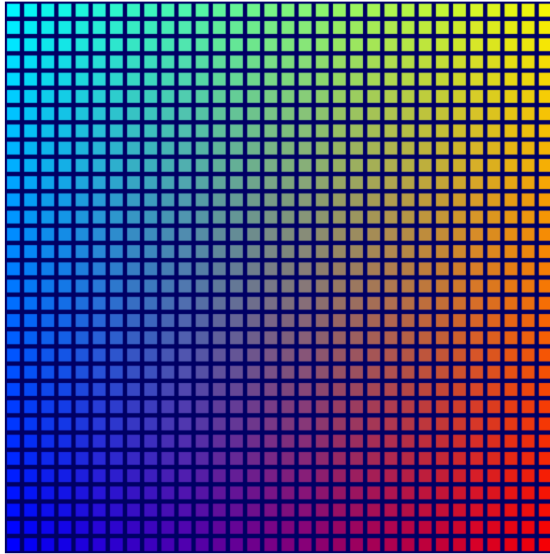
幸好，您還有一個未使用的完整色版 (藍色) 可以使用。理想效果是藍色在其他顏色最暗的地方最亮，然後隨著其他顏色強度增加而逐漸淡出。最簡單的方法是讓管道從 1 開始，然後減去其中一個儲存格值。可以是 `c.x` 或 `c.y`。不妨兩種都試試，然後選擇自己喜歡的！

6. 在片段著色器中加入更明亮的顏色，如下所示：

createShaderModule 呼叫

```
@fragment
fn fragmentMain(input: VertexOutput) -> @location(0) vec4f {
    let c = input.cell / grid;
    return vec4f(c, 1-c.x, 1);
}
```

結果看起來相當不錯！



這不是必要步驟！但因為看起來比較好，所以已納入對應的檢查點來源檔案，本程式碼研究室中的其餘螢幕截圖也反映了這個色彩更豐富的格線。

7. 管理儲存格狀態

接著，您需要根據儲存在 GPU 上的某些狀態，控管格線上的哪些儲存格會算繪。這對最終模擬至關重要！

您只需要每個儲存格的開啟/關閉信號，因此任何可儲存大量近乎任何值類型的陣列的選項都適用。您可能會認為這是統一緩衝區的另一個用途！雖然您可以讓這項功能運作，但會比較困難，因為統一緩衝區的大小有限，無法支援動態大小的陣列 (您必須在著色器中指定陣列大小)，且無法由運算著色器寫入。最後一個項目最容易出問題，因為您想在運算著色器中，於 GPU 上執行生命遊戲模擬。

幸好，還有另一種緩衝區選項，可避免上述所有限制。

建立儲存空間緩衝區

儲存緩衝區是通用緩衝區，可在運算著色器中讀取及寫入，並在頂點著色器中讀取。它們可以非常大，且不需要在著色器中宣告特定大小，因此更像一般記憶體。您會使用這個項目儲存儲存格狀態。

注意：既然儲存空間緩衝區的彈性這麼大，為什麼還要使用統一緩衝區？這其實取決於 GPU 硬體！GPU 很可能會特別處理統一緩衝區，以便更新及讀取速度比儲存緩衝區更快，因此對於可能經常更新的少量資料 (例如 3D 應用程式中的模型、檢視畫面和投影矩陣)，統一緩衝區通常是較安全的選擇，可提升效能。

1. 如要為儲存格狀態建立儲存空間緩衝區，請使用您可能已開始熟悉的緩衝區建立程式碼片段：

index.html

```
// Create an array representing the active state of each cell.
const cellStateArray = new Uint32Array(GRID_SIZE * GRID_SIZE);

// Create a storage buffer to hold the cell state.
const cellStateStorage = device.createBuffer({
  label: "Cell State",
  size: cellStateArray.byteLength,
  usage: GPUBufferUsage.STORAGE | GPUBufferUsage.COPY_DST,
});
```

與頂點和統一緩衝區一樣，請使用適當大小呼叫 `device.createBuffer()`，然後務必指定 `GPUBufferUsage.STORAGE` 的用法。

您可以填入相同大小的 `TypedArray`，然後呼叫 `device.queue.writeBuffer()`，以與先前相同的方式填入緩衝區。由於您想查看緩衝區對格線的影響，請先填入可預測的內容。

2. 使用下列程式碼啟動每第三個儲存格：

index.html

```
// Mark every third cell of the grid as active.
for (let i = 0; i < cellStateArray.length; i += 3) {
  cellStateArray[i] = 1;
}
device.queue.writeBuffer(cellStateStorage, 0, cellStateArray);
```

在著色器中讀取儲存空間緩衝區

接著，請更新著色器，在算繪格線前查看儲存空間緩衝區的內容。這與先前新增制服的方式非常相似。

1. 使用下列程式碼更新著色器：

index.html

```
@group(0) @binding(0) var<uniform> grid: vec2f;
@group(0) @binding(1) var<storage> cellState: array<u32>; // New!
```

首先，請新增繫結點，並緊貼格線制服下方。您希望保留與 `grid` 制服相同的 `@group`，但 `@binding` 號碼必須不同。`var` 型別為 `storage`，可反映緩衝區的不同型別，且您為 `cellState` 提供的型別是 `u32` 值的陣列，而非單一向量，以便與 JavaScript 中的 `Uint32Array` 相符。

注意：為什麼是 32 位元整數？如果只需要布林值，這樣做似乎很浪費資源。是啊！其實是有的！但 GPU 只能清楚顯示幾種型別的資料，因為這樣才能快速處理。舉例來說，您無法指定要公開位元組陣列，因為這與 GPU 對資料對齊方式的某些假設不相容。

現在您可以節省空間，並使用位元遮罩技巧，例如在該陣列的每個值中儲存 32 個不同儲存格的有效狀態。事實上，開發人員很早就開始想出聰明的做法，將所需資料封裝到 GPU 核准的類型中！但這樣會讓範例程式碼複雜許多。事實上，對於像這樣相對簡單的用途，您不必太擔心記憶體影響。

接著，在 `@vertex` 函式的主體中，查詢儲存格的狀態。由於狀態會儲存在儲存空間緩衝區的扁平陣列中，因此您可以使用 `instance_index` 查詢目前儲存格的值！

如果儲存格狀態顯示為「無效」，如何關閉儲存格？由於您從陣列取得的有效和無效狀態是 1 或 0，因此可以依據有效狀態縮放幾何體！如果將其縮放比例設為 1，幾何體會維持原狀；如果設為 0，幾何體會縮成單一點，GPU 隨後會捨棄該點。

2. 更新著色器程式碼，根據儲存格的啟用狀態調整位置。狀態值必須轉換為 `f32`，才能符合 WGSL 的類型安全需求：

index.html

```
@vertex
fn vertexMain(@location(0) pos: vec2f,
               @builtin(instance_index) instance: u32) -> VertexOutput {
    let i = f32(instance);
    let cell = vec2f(i % grid.x, floor(i / grid.x));
    let state = f32(cellState[instance]); // New line!

    let cellOffset = cell / grid * 2;
    // New: Scale the position by the cell's active state.
    let gridPos = (pos*state+1) / grid - 1 + cellOffset;

    var output: VertexOutput;
    output.pos = vec4f(gridPos, 0, 1);
    output.cell = cell;
    return output;
}
```

將儲存空間緩衝區新增至繫結群組

如要讓儲存緩衝區生效，請先將其新增至繫結群組。由於它與統一緩衝區屬於同一個 `@group`，因此也請將其新增至 JavaScript 程式碼中的相同繫結群組。

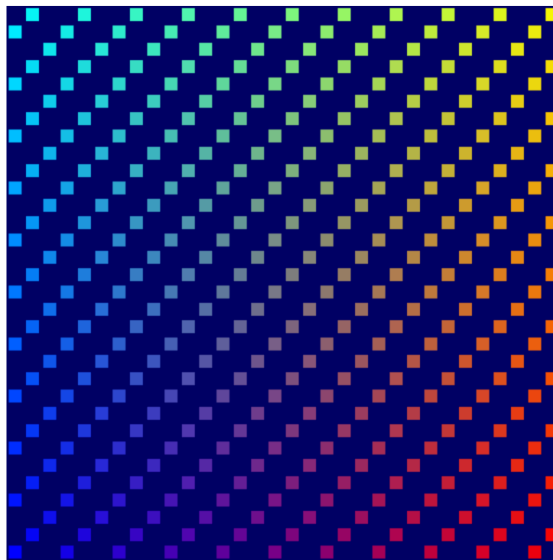
- 新增儲存空間緩衝區，如下所示：

index.html

```
const bindGroup = device.createBindGroup({
  label: "Cell renderer bind group",
  layout: cellPipeline.getBindGroupLayout(0),
  entries: [{
    binding: 0,
    resource: { buffer: uniformBuffer }
  },
  // New entry!
  {
    binding: 1,
    resource: { buffer: cellStateStorage }
  }],
});
```

請確認新項目的 `binding` 與著色器中對應值的 `@binding()` 相符！

完成上述步驟後，您應該就能重新整理，並在格線中看到模式。



使用乒乓緩衝區模式

您建構的這類模擬通常至少會使用兩個狀態副本。在模擬作業的每個步驟中，他們會從一個狀態副本讀取資料，並寫入另一個副本。接著，在下一個步驟中，翻轉並從先前寫入的狀態讀取資料。這通常稱為「乒乓」模式，因為狀態的最新版本會在每個步驟中，於狀態副本之間來回傳送。

為什麼必須這麼做？請看簡化的範例：假設您要編寫非常簡單的模擬，在每個步驟中，將所有有效方塊向右移動一個儲存格。為方便理解，您可以在 JavaScript 中定義資料和模擬：

```
// Example simulation. Don't copy into the project!
const state = [1, 0, 0, 0, 0, 0, 0, 0, 0];

function simulate() {
  for (let i = 0; i < state.length-1; ++i) {
    if (state[i] == 1) {
      state[i] = 0;
      state[i+1] = 1;
    }
  }
}

simulate(); // Run the simulation for one step.
```

但如果您執行該程式碼，作用中儲存格會一步到位，直接移到陣列結尾！這是因為因為您會持續就地更新狀態，所以會向右移動作用中儲存格，然後查看下一個儲存格，結果... 嘿！已啟用！最好還是移到右側。您在觀察資料的同時變更資料，會導致結果損毀。

使用乒乓模式可確保您一律只使用上一個步驟的結果執行模擬的下一個步驟。

```
// Example simulation. Don't copy into the project!
const stateA = [1, 0, 0, 0, 0, 0, 0, 0, 0];
const stateB = [0, 0, 0, 0, 0, 0, 0, 0, 0];

function simulate(inArray, outArray) {
  outArray[0] = 0;
  for (let i = 1; i < inArray.length; ++i) {
    outArray[i] = inArray[i-1];
  }
}

// Run the simulation for two step.
simulate(stateA, stateB);
simulate(stateB, stateA);
```

1. 更新儲存空間緩衝區分配作業，建立兩個相同的緩衝區，即可在自己的程式碼中使用這個模式：

index.html

```
// Create two storage buffers to hold the cell state.
const cellStateStorage = [
  device.createBuffer({
    label: "Cell State A",
    size: cellStateArray.byteLength,
    usage: GPUBufferUsage.STORAGE | GPUBufferUsage.COPY_DST,
  }),
  device.createBuffer({
    label: "Cell State B",
    size: cellStateArray.byteLength,
    usage: GPUBufferUsage.STORAGE | GPUBufferUsage.COPY_DST,
  })
];
```

2. 如要協助呈現兩個緩衝區的差異，請填入不同的資料：

index.html

```
// Mark every third cell of the first grid as active.
for (let i = 0; i < cellStateArray.length; i+=3) {
  cellStateArray[i] = 1;
}
device.queue.writeBuffer(cellStateStorage[0], 0, cellStateArray);

// Mark every other cell of the second grid as active.
for (let i = 0; i < cellStateArray.length; i++) {
  cellStateArray[i] = i % 2;
}
device.queue.writeBuffer(cellStateStorage[1], 0, cellStateArray);
```

注意：如上述程式碼片段所示，呼叫 `writeBuffer()` 後，您不必再保留來源 `TypedArray` 的內容。此時內容已複製完畢，且 GPU 緩衝區保證會收到呼叫時的資料。這樣您就能在下次上傳時重複使用 JavaScript 物件，節省記憶體！

3. 如要在算繪中顯示不同的儲存緩衝區，請更新繫結群組，使其也包含兩種不同的變體：

index.html


```

const bindGroups = [
  device.createBindGroup({
    label: "Cell renderer bind group A",
    layout: cellPipeline.getBindGroupLayout(0),
    entries: [{
      binding: 0,
      resource: { buffer: uniformBuffer }
    }, {
      binding: 1,
      resource: { buffer: cellStateStorage[0] }
    }],
  }),
  device.createBindGroup({
    label: "Cell renderer bind group B",
    layout: cellPipeline.getBindGroupLayout(0),
    entries: [{
      binding: 0,
      resource: { buffer: uniformBuffer }
    }, {
      binding: 1,
      resource: { buffer: cellStateStorage[1] }
    }],
  })
];

```

設定算繪迴圈

目前您只在每次重新整理頁面時繪製一次，但現在您想顯示隨時間更新的資料。如要這麼做，您需要簡單的算繪迴圈。

算繪迴圈會以特定間隔將內容繪製到畫布上，並無限重複。許多遊戲和其他想順暢呈現動畫的內容，都會使用 `requestAnimationFrame()` 函式，以與螢幕重新整理率相同的速度 (每秒 60 次) 排定回呼。

這個應用程式也可以使用該值，但在這種情況下，您可能希望更新間隔時間較長，這樣就能更輕鬆地追蹤模擬作業的執行情況。請改為自行管理迴圈，以便控制模擬更新的速率。

1. 首先，請為模擬作業選擇更新速率 (200 毫秒是不錯的選擇，但您也可以選擇較慢或較快的速率)，然後追蹤已完成的模擬步驟數量。

index.html

```

const UPDATE_INTERVAL = 200; // Update every 200ms (5 times/sec)
let step = 0; // Track how many simulation steps have been run

```

2. 然後將目前用於算繪的所有程式碼移至新函式。使用 `setInterval()` 安排函式以所需間隔重複執行。請確認函式也會更新步數，並使用該步數來選擇要繫結的兩個繫結群組。

index.html

```
// Move all of our rendering code into a function
function updateGrid() {
    step++; // Increment the step count

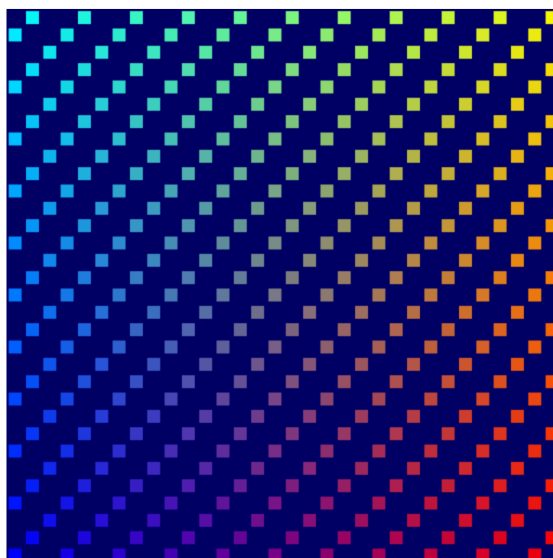
    // Start a render pass
    const encoder = device.createCommandEncoder();
    const pass = encoder.beginRenderPass({
        colorAttachments: [{
            view: context.getCurrentTexture().createView(),
            loadOp: "clear",
            clearValue: { r: 0, g: 0, b: 0.4, a: 1.0 },
            storeOp: "store",
        }]
    });

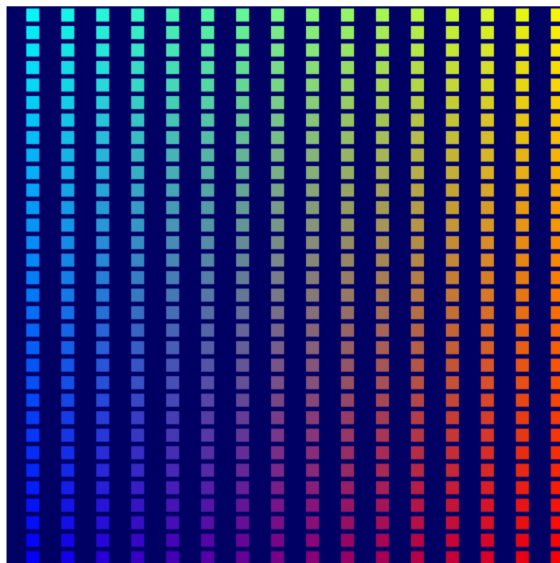
    // Draw the grid.
    pass.setPipeline(cellPipeline);
    pass.setBindGroup(0, bindGroups[step % 2]); // Updated!
    pass.setVertexBuffer(0, vertexBuffer);
    pass.draw(vertices.length / 2, GRID_SIZE * GRID_SIZE);

    // End the render pass and submit the command buffer
    pass.end();
    device.queue.submit([encoder.finish()]);
}

// Schedule updateGrid() to run repeatedly
setInterval(updateGrid, UPDATE_INTERVAL);
```

現在執行應用程式時，您會看到畫布在顯示您建立的兩個狀態緩衝區之間來回翻轉。





這樣一來，您就差不多完成算繪方面的作業了！您已準備好顯示在下一個步驟中建構的生命遊戲模擬輸出內容，並開始使用運算著色器。

顯然，WebGPU 的算繪能力遠遠超出您在此探索的一小部分，但其餘內容不在本程式碼研究室的範圍內。希望這項範例能讓您初步瞭解 WebGPU 的算繪運作方式，進而更容易掌握 3D 算繪等進階技術。

8. 執行模擬

現在，最後一塊關鍵拼圖來了：在運算著色器中執行生命遊戲模擬！

終於可以使用運算著色器了！

在本程式碼研究室中，您已抽象地瞭解運算著色器，但運算著色器究竟是什麼？

運算著色器與頂點和片段著色器類似，都是設計在 GPU 上以極度平行的方式執行，但與其他兩個著色器階段不同，運算著色器沒有特定的輸入和輸出集。您只會從所選來源 (例如儲存緩衝區) 讀取及寫入資料。也就是說，您必須告知要叫用著色器函式幾次，而不是針對每個頂點、執行個體或像素執行一次。接著，當您執行著色器時，系統會告知您正在處理哪個叫用，您可以決定要存取哪些資料，以及要從該處執行哪些作業。

運算著色器必須在著色器模組中建立，就像頂點和片段著色器一樣，因此請將其新增至程式碼，以便開始使用。如您所料，根據您實作的其他著色器結構，計算著色器的主要函式必須標示 `@compute` 屬性。

1. 使用下列程式碼建立運算著色器：

index.html

```
// Create the compute shader that will process the simulation.
const simulationShaderModule = device.createShaderModule({
  label: "Game of Life simulation shader",
  code: `
    @compute
    fn computeMain() {

    }`
});
```

由於 GPU 經常用於 3D 繪圖，因此運算著色器的結構可讓您要求沿著 X、Y 和 Z 軸呼叫著色器特定次數。這樣您就能輕鬆調度符合 2D 或 3D 格線的工作，非常適合您的使用案例！您希望呼叫這個著色器 `GRID_SIZE` 次 `GRID_SIZE` 次，模擬的每個儲存格各呼叫一次。

由於 GPU 硬體架構的性質，這個格線會劃分為工作群組。工作群組有 X、Y 和 Z 大小，雖然大小可以各為 1，但通常將工作群組稍微放大，有助於提升效能。對於著色器，請選擇 8 x 8 的任意工作群組大小。這有助於在 JavaScript 程式碼中追蹤。

2. 定義工作群組大小的常數，如下所示：

index.html

```
const WORKGROUP_SIZE = 8;
```

您也需要在著色器函式本身加入工作群組大小，方法是使用 [JavaScript 的範本常值](#)，以便輕鬆使用剛定義的常數。

3. 將工作群組大小新增至著色器函式，如下所示：

index.html (Compute createShaderModule call)

```
@compute
@workgroup_size(${WORKGROUP_SIZE}, ${WORKGROUP_SIZE}) // New line
fn computeMain() {

}
```

這會告知著色器，使用此函式完成的工作是在 $(8 \times 8 \times 1)$ 群組中完成。(如果省略任何軸，預設值為 1，但您至少必須指定 X 軸)。

附註：如要進一步使用運算著色器，工作群組大小就變得更加重要。單一工作群組中的著色器調用作業可共用快速記憶體，並使用特定類型的同步基本體。不過，由於著色器執行作業完全獨立，因此您不需要這些項目。

您可以將工作群組大小設為 $(1 \times 1 \times 1)$ ，這樣仍可正常運作，但也會限制 GPU 平行執行著色器的效能。選擇較大的值有助於 GPU 更妥善地分配工作。

每個 GPU 都有理論上的理想工作群組大小，但這取決於 WebGPU 不會公開的架構詳細資料，因此通常會根據著色器的需求選擇數字。如果沒有，考量到 WebGPU 內容可能執行的硬體範圍很廣，64 是個不錯的數字，不太可能超出任何硬體限制，但仍可處理足夠大的批次，達到合理的效率。 $(8 \times 8 == 64)$ ，因此工作群組大小會遵循這項建議)。

與其他著色器階段一樣，您可以接受各種 `@builtin` 值做為運算著色器函式的輸入內容，藉此瞭解您目前執行的叫用，並決定需要執行的工作。

4. 新增 `@builtin` 值，如下所示：

index.html (Compute createShaderModule call)

```
@compute @workgroup_size(${WORKGROUP_SIZE}, ${WORKGROUP_SIZE})
fn computeMain(@builtin(global_invocation_id) cell: vec3u) {

}
```

您傳遞的是 `global_invocation_id` 內建函式，這是無正負號整數的三維向量，可告知您在著色器調用網格中的位置。您會針對格線中的每個儲存格執行一次這個著色器。您會取得 $(0, 0, 0)$ 、 $(1, 0, 0)$ 、 $(1, 1, 0)$... 一直到 $(31, 31, 0)$ 的數字，這表示您可以將其視為要執行的儲存格索引！

計算著色器也可以使用制服，就像在頂點和片段著色器中一樣。

5. 使用與計算著色器相同的 Uniform，告知格線大小，如下所示：

index.html (Compute createShaderModule call)

```
@group(0) @binding(0) var<uniform> grid: vec2f; // New line

@compute @workgroup_size(${WORKGROUP_SIZE}, ${WORKGROUP_SIZE})
fn computeMain(@builtin(global_invocation_id) cell: vec3u) {

}
```

與頂點著色器相同，您也會將儲存格狀態公開為儲存空間緩衝區。但在此情況下，您需要兩個！由於運算著色器沒有必要輸出內容 (例如頂點位置或片段顏色)，因此將值寫入儲存緩衝區或紋理，是從運算著色器取得結果的唯一方法。使用您先

前學過的乒乓方法；您有一個儲存空間緩衝區，可饋送格線的目前狀態，另一個則可寫出格線的新狀態。

6. 將儲存格輸入和輸出狀態公開為儲存緩衝區，如下所示：

index.html (Compute createShaderModule call)

```
@group(0) @binding(0) var<uniform> grid: vec2f;

// New lines
@group(0) @binding(1) var<storage> cellStateIn: array<u32>;
@group(0) @binding(2) var<storage, read_write> cellStateOut: array<u32>;

@compute @workgroup_size(${WORKGROUP_SIZE}, ${WORKGROUP_SIZE})
fn computeMain(@builtin(global_invocation_id) cell: vec3u) {

}
```

請注意，第一個儲存空間緩衝區是以 `var<storage>` 宣告，因此為唯讀，但第二個儲存空間緩衝區是以 `var<storage, read_write>` 宣告。這樣一來，您就能讀取及寫入緩衝區，並將該緩衝區做為運算著色器的輸出內容。(WebGPU 沒有僅供寫入的儲存空間模式)。

接下來，您需要將儲存格索引對應至線性儲存陣列。這基本上與您在頂點著色器中執行的操作相反，您當時是採用線性 `instance_index` 並將其對應至 2D 格線儲存格。(提醒您，該演算法為 `vec2f(i % grid.x, floor(i / grid.x))`)。

7. 撰寫函式，朝另一個方向移動。這個函式會取得儲存格的 Y 值，乘以格線寬度，然後加上儲存格的 X 值。

index.html (Compute createShaderModule call)

```
@group(0) @binding(0) var<uniform> grid: vec2f;

@group(0) @binding(1) var<storage> cellStateIn: array<u32>;
@group(0) @binding(2) var<storage, read_write> cellStateOut: array<u32>;

// New function
fn cellIndex(cell: vec2u) -> u32 {
    return cell.y * u32(grid.x) + cell.x;
}

@compute @workgroup_size(${WORKGROUP_SIZE}, ${WORKGROUP_SIZE})
fn computeMain(@builtin(global_invocation_id) cell: vec3u) {

}
```

最後，為了確認演算法是否正常運作，請實作非常簡單的演算法：如果儲存格目前為開啟狀態，則關閉；反之亦然。這還不是生命遊戲，但足以證明運算著色器正常運作。

8. 新增簡單的演算法，如下所示：

index.html (Compute createShaderModule call)

```

@group(0) @binding(0) var<uniform> grid: vec2f;

@group(0) @binding(1) var<storage> cellStateIn: array<u32>;
@group(0) @binding(2) var<storage, read_write> cellStateOut: array<u32>;

fn cellIndex(cell: vec2u) -> u32 {
    return cell.y * u32(grid.x) + cell.x;
}

@compute @workgroup_size(${WORKGROUP_SIZE}, ${WORKGROUP_SIZE})
fn computeMain(@builtin(global_invocation_id) cell: vec3u) {
    // New lines. Flip the cell state every step.
    if (cellStateIn[cellIndex(cell.xy)] == 1) {
        cellStateOut[cellIndex(cell.xy)] = 0;
    } else {
        cellStateOut[cellIndex(cell.xy)] = 1;
    }
}

```

注意：這裡的 `cell.xy` 語法是簡寫，又稱為「swizzling」。這相當於說 `vec2(cell.x, cell.y)`，也是從不同設定的向量中取得所需元件的簡單方式。Swizzling 非常靈活！您可以依任意順序放置向量的元件（最多四個），並視需要重複使用，例如 `cell.zyx` 或 `cell.yyyy`。

目前為止，運算著色器就是這樣！不過，您還需要進行幾項變更，才能查看結果。

使用繫結群組和管道版面配置

您可能會發現，上述著色器大多使用與算繪管道相同的輸入內容（統一和儲存空間緩衝區）。因此您可能會認為只要使用相同的繫結群組即可，對吧？好消息是，你可以！只要多花一點時間手動設定，就能達成這個目標。

每次建立繫結群組時，都必須提供 `GPUBindGroupLayout`。先前，您是在算繪管道上呼叫 `getBindGroupLayout()` 取得該版面配置，而系統會自動建立該版面配置，因為您在建立時提供了 `layout: "auto"`。如果您只使用單一管道，這種做法很有效，但如果您有多個管道要共用資源，就必須明確建立版面配置，然後提供給繫結群組和管道。

為瞭解原因，請考慮以下情況：您在算繪管道中使用單一統一緩衝區和單一儲存緩衝區，但在您剛編寫的運算著色器中，需要第二個儲存緩衝區。由於這兩個著色器對統一和第一個儲存緩衝區使用相同的 `@binding` 值，因此您可以在管道之間共用這些值，而算繪管道會忽略第二個儲存緩衝區，因為管道不會使用該緩衝區。您想建立的版面配置會描述繫結群組中的「所有」資源，而不只是特定管道使用的資源。

1. 如要建立該版面配置，請呼叫 `device.createBindGroupLayout()`：

`index.html`

```
// Create the bind group layout and pipeline layout.
const bindGroupLayout = device.createBindGroupLayout({
  label: "Cell Bind Group Layout",
  entries: [{
    binding: 0,
    // Add GPUShaderStage.FRAGMENT here if you are using the `grid` uniform in the fragment
    // shader.
    visibility: GPUShaderStage.VERTEX | GPUShaderStage.COMPUTE,
    buffer: {} // Grid uniform buffer
  }, {
    binding: 1,
    visibility: GPUShaderStage.VERTEX | GPUShaderStage.COMPUTE,
    buffer: { type: "read-only-storage" } // Cell state input buffer
  }, {
    binding: 2,
    visibility: GPUShaderStage.COMPUTE,
    buffer: { type: "storage" } // Cell state output buffer
  }]
});
```

這與建立繫結群組本身的結構類似，因為您會描述 `entries` 清單。不同之處在於，您要說明項目必須是哪種資源，以及如何使用，而不是提供資源本身。

在每個項目中，您會提供資源的 `binding` 編號 (如您在建立繫結群組時所學，這與著色器中的 `@binding` 值相符)。您也會提供 `visibility`，也就是 `GPUShaderStage` 旗標，指出哪些著色器階段可以使用資源。您希望在頂點和運算著色器中存取統一和第一個儲存空間緩衝區，但第二個儲存空間緩衝區只需要在運算著色器中存取。

注意：如果片段著色器程式碼使用 uniform `grid` 變數 (如上一節所述)，您可能需要新增 `GPUShaderStage.FRAGMENT`，讓片段著色器存取資源 (適用於 uniform 和第一個儲存空間緩衝區)。

最後，請指出使用的資源類型。視要公開的內容而定，這會是不同的字典鍵。這裡的三個資源都是緩衝區，因此您可以使用 `buffer` 鍵定義每個資源的選項。其他選項包括 `texture` 或 `sampler` 等，但這裡不需要這些選項。

在緩衝區字典中，您可以設定緩衝區選項，例如要使用哪種緩衝區 `type`。預設值為 `"uniform"`，因此您可以將字典留空，以繫結 0。(不過，您至少必須設定 `buffer: {}`，才能將項目識別為緩衝區)。由於您未使用著色器中的 `read_write` 存取權，因此繫結 1 的類型為 `"read-only-storage"`，而繫結 2 的類型為 `"storage"`，因為您確實使用了 `read_write` 存取權！

建立 `bindGroupLayout` 後，您可以在建立繫結群組時傳遞該物件，不必從管道查詢繫結群組。這麼做表示您必須在每個繫結群組中新增儲存空間緩衝區項目，才能與剛定義的版面配置相符。

2. 更新繫結群組建立作業，如下所示：

`index.html`


```
// Create a bind group to pass the grid uniforms into the pipeline
const bindGroups = [
  device.createBindGroup({
    label: "Cell renderer bind group A",
    layout: bindGroupLayout, // Updated Line
    entries: [{
      binding: 0,
      resource: { buffer: uniformBuffer }
    }, {
      binding: 1,
      resource: { buffer: cellStateStorage[0] }
    }, {
      binding: 2, // New Entry
      resource: { buffer: cellStateStorage[1] }
    }],
  }),
  device.createBindGroup({
    label: "Cell renderer bind group B",
    layout: bindGroupLayout, // Updated Line

    entries: [{
      binding: 0,
      resource: { buffer: uniformBuffer }
    }, {
      binding: 1,
      resource: { buffer: cellStateStorage[1] }
    }, {
      binding: 2, // New Entry
      resource: { buffer: cellStateStorage[0] }
    }],
  }),
];
```

現在繫結群組已更新為使用這個明確的繫結群組版面配置，您需要更新算繪管道，以使用相同的項目。

3. 建立 `GPUPipelineLayout` 。

index.html

```
const pipelineLayout = device.createPipelineLayout({
  label: "Cell Pipeline Layout",
  bindGroupLayouts: [ bindGroupLayout ],
});
```

管道版面配置是管道使用的一或多個繫結群組版面配置清單 (在本例中，您有一個)。陣列中的繫結群組版面配置順序必須與著色器中的 `@group` 屬性相符。(這表示 `bindGroupLayout` 與 `@group(0)` 相關聯)。

4. 取得管道版面配置後，請更新算繪管道，改用該版面配置，而非 `"auto"` 。

index.html

```
const cellPipeline = device.createRenderPipeline({
  label: "Cell pipeline",
  layout: pipelineLayout, // Updated!
  vertex: {
    module: cellShaderModule,
    entryPoint: "vertexMain",
    buffers: [vertexBufferLayout]
  },
  fragment: {
    module: cellShaderModule,
    entryPoint: "fragmentMain",
    targets: [{
      format: canvasFormat
    }]
  }
});
```

建立運算管道

就像使用頂點和片段著色器需要算繪管線一樣，使用運算著色器也需要運算管線。幸好，運算管道比算繪管道簡單得多，因為運算管道不需要設定任何狀態，只需要著色器和版面配置。

- 使用下列程式碼建立運算管線：

index.html

```
// Create a compute pipeline that updates the game state.
const simulationPipeline = device.createComputePipeline({
  label: "Simulation pipeline",
  layout: pipelineLayout,
  compute: {
    module: simulationShaderModule,
    entryPoint: "computeMain",
  }
});
```

請注意，您傳遞的是新的 `pipelineLayout`，而不是 `"auto"`，就像在更新後的算繪管道中一樣，這可確保算繪管道和運算管道都能使用相同的繫結群組。

運算通行證

這時您就可以實際使用運算管道了！由於您是在算繪通道中進行算繪，因此您可能可以猜到，您需要在運算通道中執行運算工作。運算和算繪工作都可以在同一個指令編碼器中進行，因此您需要稍微改動 `updateGrid` 函式。

1. 將編碼器建立作業移至函式頂端，然後使用該編碼器啟動運算傳遞 (在 `step++` 之前)。

index.html

```
// In updateGrid()
// Move the encoder creation to the top of the function.
const encoder = device.createCommandEncoder();

const computePass = encoder.beginComputePass();

// Compute work will go here...

computePass.end();

// Existing lines
step++; // Increment the step count

// Start a render pass...
```

與運算管道一樣，運算傳遞的啟動方式比對應的算繪傳遞簡單許多，因為您不必擔心任何附件。

您希望在算繪通道之前執行運算通道，因為這樣算繪通道就能立即使用運算通道的最新結果。這也是您在傳遞之間遞增 `step` 計數的原因，這樣一來，運算管線的輸出緩衝區就會成為算繪管線的輸入緩衝區。

2. 接著，在運算傳遞中設定管道和繫結群組，並使用與算繪傳遞相同的模式，在繫結群組之間切換。

index.html

```
const computePass = encoder.beginComputePass();

// New lines
computePass.setPipeline(simulationPipeline);
computePass.setBindGroup(0, bindGroups[step % 2]);

computePass.end();
```

3. 最後，您會將工作調度至運算著色器，並告知每個軸要執行的工作群組數量，而不是像在算繪通道中一樣繪製。

index.html

```
const computePass = encoder.beginComputePass();

computePass.setPipeline(simulationPipeline);
computePass.setBindGroup(0, bindGroups[step % 2]);

// New lines
const workgroupCount = Math.ceil(GRID_SIZE / WORKGROUP_SIZE);
computePass.dispatchWorkgroups(workgroupCount, workgroupCount);

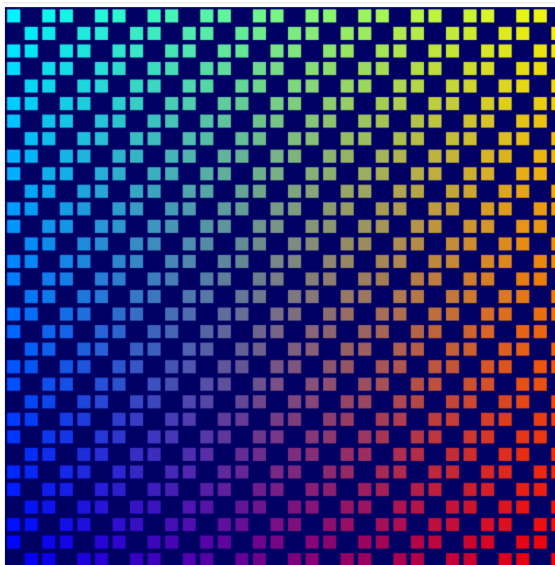
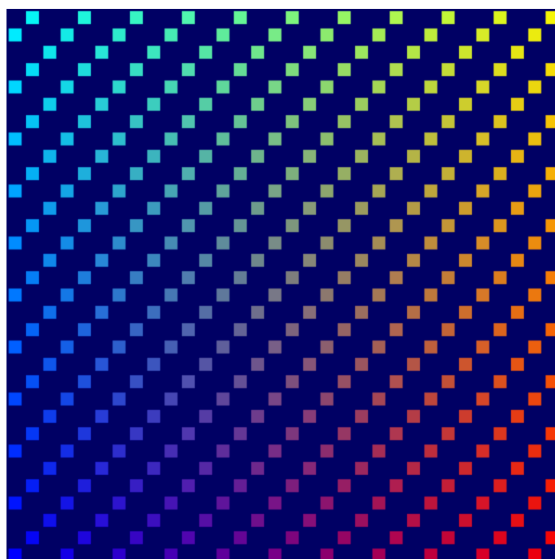
computePass.end();
```

請注意，您傳遞至 `dispatchWorkgroups()` 的數字並非叫用次數，這點非常重要！而是要執行的工作群組數量，如著色器中的 `@workgroup_size` 所定義。

如果想讓著色器執行 32x32 次，以涵蓋整個格線，且工作群組大小為 8x8，則需要調度 4x4 工作群組 ($4 * 8 = 32$)。因此，您要將格線大小除以工作群組大小，然後將該值傳遞至 `dispatchWorkgroups()`。

注意：運算工作只能由工作群組調度，因此如果工作負載無法平均分配給工作群組，您有兩種選擇。您可以變更工作群組大小 (必須在著色器模組建立時完成，且負擔相對較高)，也可以將您調度的工作群組數量向上取整，然後在著色器中檢查是否超過所需 `global_invocation_id`，並視需要提早傳回。

現在您可以再次重新整理頁面，應該會看到格線在每次更新時反轉。



實作生命遊戲的演算法

在更新運算著色器以實作最終演算法之前，請先返回初始化儲存空間緩衝區內容的程式碼，並更新該程式碼，以便在每次載入網頁時產生隨機緩衝區。(規律的模式並非生命遊戲的有趣起點)。您可以隨意隨機產生值，但我們提供簡單的入門方法，可產生合理的結果。

1. 如要讓每個儲存格都以隨機狀態啟動，請將 `cellStateArray` 初始化更新為下列程式碼：

`index.html`

```
// Set each cell to a random state, then copy the JavaScript array
// into the storage buffer.
for (let i = 0; i < cellStateArray.length; ++i) {
    cellStateArray[i] = Math.random() > 0.6 ? 1 : 0;
}
device.queue.writeBuffer(cellStateStorage[0], 0, cellStateArray);
```

現在，您終於可以實作生命遊戲模擬的邏輯。歷經千辛萬苦，著色器程式碼可能簡單得令人失望！

首先，您需要知道任何指定儲存格有多少個鄰近儲存格處於啟用狀態。您不在意哪些帳戶有效，只在意數量。

2. 若要更輕鬆地取得相鄰儲存格資料，請新增 `cellActive` 函式，傳回指定座標的 `cellStateIn` 值。

index.html (Compute createShaderModule call)

```
fn cellActive(x: u32, y: u32) -> u32 {
    return cellStateIn[cellIndex(vec2(x, y))];
}
```

如果儲存格處於啟用狀態，`cellActive` 函式會傳回 1，因此將所有八個周圍儲存格的 `cellActive` 呼叫傳回值相加，即可得知有多少相鄰儲存格處於啟用狀態。

3. 找出活躍鄰居人數，如下所示：

index.html (Compute createShaderModule call)

```
fn computeMain(@builtin(global_invocation_id) cell: vec3u) {
    // New lines:
    // Determine how many active neighbors this cell has.
    let activeNeighbors = cellActive(cell.x+1, cell.y+1) +
                          cellActive(cell.x+1, cell.y) +
                          cellActive(cell.x+1, cell.y-1) +
                          cellActive(cell.x, cell.y-1) +
                          cellActive(cell.x-1, cell.y-1) +
                          cellActive(cell.x-1, cell.y) +
                          cellActive(cell.x-1, cell.y+1) +
                          cellActive(cell.x, cell.y+1);
```

不過，這會導致一個小問題：如果檢查的儲存格位於棋盤邊緣，會發生什麼情況？根據您目前的 `cellIndex()` 邏輯，這會溢位至下一個或上一個資料列，或是超出緩衝區邊緣！

注意：與某些語言不同，在 WGSL 中，[超出緩衝區界限](#)的索引並非不安全，但無法預測。語言會確保您仍可從緩衝區取得部分值，因此您不會意外讀取其他程序中的資料或類似內容，但幾乎可以確定這不會是您想要的結果。此外，行為可能會因使用的平台和瀏覽器而異，因此您絕不應依賴超出範圍的存取權。

以生命遊戲為例，解決這個問題的常見簡單方法是讓格線邊緣的儲存格將格線對向邊緣的儲存格視為鄰居，產生環繞效果。

4. 只要稍微變更 `cellIndex()` 函式，即可支援格線環繞。

index.html (Compute createShaderModule call)

```
fn cellIndex(cell: vec2u) -> u32 {
    return (cell.y % u32(grid.y)) * u32(grid.x) +
        (cell.x % u32(grid.x));
}
```

使用 `%` 運算子包裝超出格線大小的儲存格 `X` 和 `Y` 時，可確保您絕不會存取儲存緩衝區界限外的內容。因此，您可以放心，`activeNeighbors` 計數是可預測的。

然後套用下列其中一項規則：

- 任何鄰居少於兩個的儲存格都會變成非使用中。
- 如果有效儲存格有兩個或三個相鄰儲存格，則會保持有效。
- 如果閒置儲存格正好有三個鄰居，就會變成啟用狀態。
- 如果儲存格有超過三個相鄰儲存格，就會變成無效。

您可以使用一系列 `if` 陳述式執行這項操作，但 WGSL 也支援 `switch` 陳述式，非常適合這個邏輯。

5. 實作生命遊戲邏輯，如下所示：

index.html (Compute createShaderModule call)

```
let i = cellIndex(cell.xy);

// Conway's game of life rules:
switch activeNeighbors {
    case 2: { // Active cells with 2 neighbors stay active.
        cellStateOut[i] = cellStateIn[i];
    }
    case 3: { // Cells with 3 neighbors become or stay active.
        cellStateOut[i] = 1;
    }
    default: { // Cells with < 2 or > 3 neighbors become inactive.
        cellStateOut[i] = 0;
    }
}
```

供您參考，最終的運算著色器模組呼叫現在如下所示：

index.html

```

const simulationShaderModule = device.createShaderModule({
  label: "Life simulation shader",
  code: `
    @group(0) @binding(0) var<uniform> grid: vec2f;

    @group(0) @binding(1) var<storage> cellStateIn: array<u32>;
    @group(0) @binding(2) var<storage, read_write> cellStateOut: array<u32>;

    fn cellIndex(cell: vec2u) -> u32 {
      return (cell.y % u32(grid.y)) * u32(grid.x) +
        (cell.x % u32(grid.x));
    }

    fn cellActive(x: u32, y: u32) -> u32 {
      return cellStateIn[cellIndex(vec2(x, y))];
    }

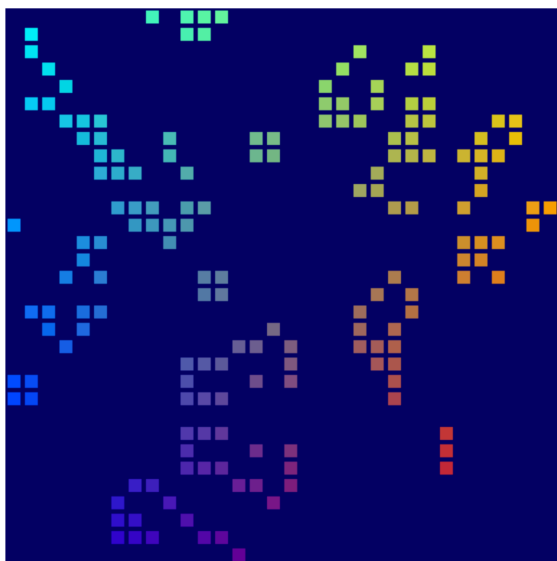
    @compute @workgroup_size(${WORKGROUP_SIZE}, ${WORKGROUP_SIZE})
    fn computeMain(@builtin(global_invocation_id) cell: vec3u) {
      // Determine how many active neighbors this cell has.
      let activeNeighbors = cellActive(cell.x+1, cell.y+1) +
        cellActive(cell.x+1, cell.y) +
        cellActive(cell.x+1, cell.y-1) +
        cellActive(cell.x, cell.y-1) +
        cellActive(cell.x-1, cell.y-1) +
        cellActive(cell.x-1, cell.y) +
        cellActive(cell.x-1, cell.y+1) +
        cellActive(cell.x, cell.y+1);

      let i = cellIndex(cell.xy);

      // Conway's game of life rules:
      switch activeNeighbors {
        case 2: {
          cellStateOut[i] = cellStateIn[i];
        }
        case 3: {
          cellStateOut[i] = 1;
        }
        default: {
          cellStateOut[i] = 0;
        }
      }
    }
  `;
});

```

這樣就完成了！大功告成！重新整理頁面，即可看到新建立的細胞自動機成長！



步驟 9 · 約 0 分鐘

9. 恭喜！

您使用 WebGPU API 建立的經典康威生命遊戲模擬版本，完全在 GPU 上執行！

後續步驟

- 查看 [WebGPU 範例](#)。

其他資訊

- [WebGPU](#)：使用所有核心，但沒有畫布
- [原始 WebGPU](#)
- [WebGPU 基礎知識](#)
- [WebGPU 最佳做法](#)

參考文件

- [WebGPU 規格](#)
 - [WGSL 規格](#)
-